

POSM qFoundry

Technical Design and Implementation Specification

Companion Volume to the POSM qFoundry System Specification

Document purpose. This volume defines how qFoundry is to be implemented. It specifies code boundaries, process topology, data models, durable workflows, model adapters, cost enforcement, statistical systems, user interfaces, security controls, deployment, and verification gates at implementation depth.

Version 0.1-TECHNICAL-DRAFT

June 2026

Photonic and Optoelectronic Systems and Methods (POSM)

Private engineering draft. Intended for POSM implementation review before public release.

Document Control

Document	POSM qFoundry Technical Design and Implementation Specification
Status	Technical draft for architecture review and capability-spike implementation
Relationship	Companion to the general POSM qFoundry System Specification v0.1
Normative language	“Shall” and “must” are mandatory; “should” is recommended; “may” is optional
Primary audience	POSM engineering leads, implementers, reviewers, faculty advisers, and future open-source maintainers
Baseline deployment	Single-host Windows/Tauri client with a Linux execution layer; later dedicated Linux lab server and multiple desktop clients
Primary coding worker	Claude Agent SDK, Sonnet by default and Opus under policy
Primary supervisor	OpenAI reasoning model through the OpenAI Agents SDK
Durability	Temporal workflows plus PostgreSQL transactional state
License target	AGPLv3 for software; separate POSM trademark policy

Change discipline

This technical specification is version controlled. Any change that affects a safety invariant, user-visible contract behavior, cost guarantee, data-retention rule, or externally observable API shall be represented by an architecture decision record and a corresponding change request in the product contract. Implementation pull requests shall cite the sections they satisfy.

Reading strategy

Readers implementing a specific subsystem should begin with [chapter 4](#), then read the subsystem chapter and its normative appendix. Cross-cutting requirements in budgeting, security, event provenance, and multi-tenancy apply even when they are not repeated in every subsystem chapter.

Contents

Document Control	i
I Technical Baseline	1
1 Purpose, Scope, and Conformance	2
1.1 Purpose of this volume	2
1.2 Scope	2
1.3 Conformance classes	2
1.4 Implementation philosophy	3
2 Locked Product and Technical Decisions	4
2.1 Product identity and release	4
2.2 POSM deployment policy	4
2.3 Worker and supervisor roles	4
2.4 Repository operations	4
2.5 Cost semantics	5
2.6 Human communication	5
2.7 Sentinel authority	5
3 Architecture Drivers and Quality Attributes	6
3.1 Priority order	6
3.2 Reliability objectives	6
3.3 Security objectives	6
3.4 Cost accuracy objectives	6
3.5 Observability objectives	7
3.6 Performance objectives	7
3.7 Evolvability objectives	7
4 Reference Architecture	8
4.1 System topology	8
4.2 Technology baseline	8
4.3 Modular-monolith boundary	9
4.4 Deployment evolution	9

II	Codebase and Runtime Architecture	10
5	Monorepo and Build Architecture	11
5.1	Repository organization	11
5.2	Dependency rules	12
5.3	Build and test commands	12
5.4	Configuration	12
5.5	Versioning	12
6	Control Plane Services	13
6.1	Responsibilities	13
6.2	Process decomposition	13
6.3	Command handling	13
6.4	Read model	14
6.5	Consistency	14
7	Desktop Client Architecture	15
7.1	Application model	15
7.2	Frontend stack	15
7.3	Data transport	15
7.4	Client state	15
7.5	Information density	16
7.6	Emergency controls	16
7.7	Offline and reconnect behavior	16
8	Worker Node and Executor Architecture	17
8.1	Node agent	17
8.2	Worker lease	17
8.3	Container construction	17
8.4	Mounted filesystem	17
8.5	Resource envelope	18
8.6	Command telemetry	18
8.7	Operator takeover	18
9	Durable Workflow Runtime	19
9.1	Why Temporal is authoritative for execution state	19
9.2	Workflow hierarchy	19
9.3	Determinism rule	19
9.4	Signals and queries	19
9.5	Idempotent activities	20
9.6	Continue-as-new	20
9.7	Cancellation and interruption	20
10	OpenAI Supervisor Runtime	21
10.1	Supervisor responsibilities	21
10.2	SDK use	21
10.3	Supervisor tools	21
10.4	Structured decision output	21

10.5 Prompt assembly	22
10.6 Guardrails	22
10.7 Discovery mode	22
10.8 Failure behavior	22
11 Claude Coding Worker Adapter	23
11.1 Adapter boundary	23
11.2 Session lifecycle	23
11.3 Why <code>dontAsk</code>	24
11.4 Message normalization	24
11.5 Permission callback	24
11.6 Worker result	24
11.7 Context management	24
12 Contract Compiler	26
12.1 Compiler objective	26
12.2 Compiler pipeline	26
12.3 Intermediate representation	26
12.4 Contradiction detection	27
12.5 Readiness score	27
12.6 Packet composition	27
12.7 Compiler tests	27
13 Policy and Permission Engine	28
13.1 Design	28
13.2 Policy inputs	28
13.3 Evaluation order	28
13.4 Command classification	29
13.5 Dependency approval	29
13.6 Policy language	29
13.7 Explainability	29
14 Model Gateway	30
14.1 Purpose	30
14.2 Provider abstraction	30
14.3 Model catalog	30
14.4 Request admission	31
14.5 Redaction	31
14.6 Usage normalization	31
14.7 Fallbacks	31
15 Budget Ledger and Hard-Stop Enforcement	32
15.1 Accounting model	32
15.2 Account hierarchy	32
15.3 Reservation formula	32
15.4 Admission inequality	33
15.5 Target and tolerance	33
15.6 Reconciliation	33

15.7	Threshold actions	33
15.8	Salvage funding	33
16	Preflight Estimation and Quoting	34
16.1	Purpose	34
16.2	Feature extraction	34
16.3	Cold-start estimation	34
16.4	Candidate plans	34
16.5	Quote object	34
16.6	Proof-before-build	35
16.7	Calibration	35
16.8	Quote expiry	35
17	Portfolio Scheduler and Digital Twin	36
17.1	Scheduling problem	36
17.2	Hard constraints	36
17.3	Objective	36
17.4	Two-stage scheduling	36
17.5	Fairness	36
17.6	Digital twin simulation	37
17.7	Simulation implementation	37
17.8	Schedule repair	37
17.9	User interface	37
18	Sentinel Statistical and Improvement System	38
18.1	Separation of telemetry and inference	38
18.2	Service architecture	38
18.3	Initial model portfolio	38
18.3.1	Hierarchical cost model	38
18.3.2	Completion-time model	39
18.3.3	Progress-state model	39
18.3.4	Changepoint detection	39
18.3.5	Routing model	39
18.3.6	Safe tuning	39
18.4	No hand-wavy probabilities	39
18.5	Model lifecycle	39
18.6	Improvement experiments	39
18.7	Security monitoring	40
19	Progress and Evidence Engine	41
19.1	Problem statement	41
19.2	Evidence graph	41
19.3	Progress dimensions	41
19.4	Evidence staleness	42
19.5	Meaningful progress event	42
19.6	Stagnation detection	42
19.7	Completion	42

III	Data, Interfaces, and Integrations	43
20	Domain Model and Persistence	44
20.1	Aggregate boundaries	44
20.2	Identifier policy	44
20.3	Project state machine	44
20.4	Contract versioning	44
20.5	Multi-tenancy	45
20.6	Transactional event append	45
20.7	Retention classes	45
21	Event Store, Telemetry, and Provenance	46
21.1	Two telemetry layers	46
21.2	Event envelope	46
21.3	Append-only rule	46
21.4	Sequence assignment	46
21.5	Event severity and classification	46
21.6	Live delivery	47
21.7	Raw tool output	47
21.8	Metrics	47
21.9	Trace correlation	47
22	API and Inter-Process Contracts	48
22.1	External API style	48
22.2	Core routes	48
22.3	Error model	48
22.4	Idempotency	49
22.5	Node protocol	49
22.6	Sentinel protocol	49
22.7	Schema compatibility	49
23	Git, Repository, and Artifact Subsystems	50
23.1	Repository modes	50
23.2	Worktree strategy	50
23.3	Protected operations	50
23.4	Git event capture	50
23.5	Artifact service	50
23.6	Deliverable bundle	50
23.7	Repository deletion	51
24	Email and Notification Integration	52
24.1	Dedicated identity	52
24.2	Outbound message object	52
24.3	Gmail integration	52
24.4	Reply correlation	52
24.5	Interpretation and confirmation	52
24.6	Communication windows	52
24.7	Desktop notifications	53

IV	Security and Multi-Tenancy	54
25	Identity, Authorization, and Tenant Isolation	55
25.1	Authentication	55
25.2	Roles	55
25.3	Authorization model	55
25.4	Master account	55
25.5	User project rule	55
25.6	Resource quotas	56
25.7	Immediate abuse response	56
25.8	Audit	56
26	Secrets and Credential Broker	57
26.1	Trust boundary	57
26.2	Storage	57
26.3	Brokered operations	57
26.4	Ephemeral project secrets	57
26.5	Redaction pipeline	57
26.6	Leak response	58
26.7	Backups	58
27	Sandbox, Network, and Dependency Security	59
27.1	Isolation layers	59
27.2	Network default deny	59
27.3	Domain grants	59
27.4	Package installation	59
27.5	Prompt injection from repository content	59
27.6	Canary repository	60
27.7	Windows development versus Linux production	60
28	Security Operations and Incident Response	61
28.1	Threat model maintenance	61
28.2	Incident classes	61
28.3	Evidence preservation	61
28.4	Security release gates	61
28.5	Physical controls	62
28.6	Vulnerability response	62
V	Deployment, Reliability, and Verification	63
29	Deployment and Installation	64
29.1	Profiles	64
29.2	Development profile	64
29.3	POSM single-host profile	64
29.4	POSM lab-server profile	64
29.5	Installer behavior	64
29.6	Configuration validation	64

29.7 Upgrades	65
29.8 Public packaging	65
30 Reliability, Backup, and Recovery	66
30.1 Failure domains	66
30.2 Checkpoint policy	66
30.3 Database backup	66
30.4 Artifact backup	66
30.5 Recovery objectives	66
30.6 Provider outage	67
30.7 Node outage	67
30.8 Disk full	67
30.9 Disaster recovery drill	67
31 Observability, Health, and Service Objectives	68
31.1 Health model	68
31.2 Service-level objectives	68
31.3 Logging	68
31.4 Traces	69
31.5 Alerting	69
31.6 Dashboard provenance	69
32 Performance and Compute Sizing	70
32.1 Workload characteristics	70
32.2 Per-worker planning envelope	70
32.3 Initial dedicated machine	70
32.4 Heterogeneous expansion	70
32.5 Raspberry Pi role	70
32.6 Local GPU	70
32.7 Capacity admission	71
33 Test Strategy and Verification Gates	72
33.1 Test pyramid	72
33.2 Fake providers	72
33.3 Core capability gates	72
33.4 Budget tests	72
33.5 Temporal tests	73
33.6 Security tests	73
33.7 UI tests	73
33.8 Statistical tests	73
34 Implementation Roadmap	74
34.1 Principle	74
34.2 Phase 0 - Repository and foundations	74
34.3 Phase 1 - One closed loop	74
34.4 Phase 2 - Contract and evidence	74
34.5 Phase 3 - Durable operations	74
34.6 Phase 4 - POSM multi-user	74

34.7 Phase 5 - Full dashboard	75
34.8 Phase 6 - Portfolio execution	75
34.9 Phase 7 - Sentinel advisory	75
34.10Phase 8 - Sentinel bounded authority	75
34.11Phase 9 - Public release	75
34.12Milestone discipline	75
35 Definition of Done and Release Acceptance	76
35.1 Subsystem definition of done	76
35.2 V0.1 acceptance	76
35.3 Financial acceptance	76
35.4 Security acceptance	76
35.5 Sentinel acceptance	77
35.6 Human acceptance	77
35.7 Final release package	77
VI Normative Implementation Appendices	78
A Normative PostgreSQL Schema Excerpts	79
A.1 Tenants, users, and projects	79
A.2 One active project per ordinary user	80
A.3 Contracts and requirements	80
A.4 Budget ledger	80
A.5 Events and outbox	81
A.6 Permissions and decisions	82
A.7 Row-level security example	82
B Normative TypeScript Contracts	84
B.1 Opaque identifiers	84
B.2 Money	84
B.3 Project aggregate	84
B.4 Model request	85
B.5 Normalized usage	85
B.6 Node assignment	86
B.7 Event contract	86
B.8 Permission result	86
C Workflow Pseudocode	88
C.1 Project lifecycle	88
C.2 Milestone loop	89
C.3 Permission handling	90
C.4 Salvage	90
C.5 Idempotent email activity	90
D Event Catalog	91
D.1 Project and contract events	91
D.2 Supervisor and worker events	91

D.3	Budget events	91
D.4	Verification and progress events	92
D.5	Security and infrastructure events	92
E	Permission Rules and Command Classes	93
E.1	Global invariants	93
E.2	Green actions	93
E.3	Yellow actions	93
E.4	Red actions	94
E.5	Command parsing examples	94
F	Budget Algorithms and Numerical Examples	95
F.1	Fixed-point arithmetic	95
F.2	Atomic reservation algorithm	95
F.3	Settlement	96
F.4	Example: target with tolerance	96
F.5	Reservation utilization	96
F.6	Project threshold state	96
F.7	Provider cap mismatch	96
G	Sentinel Model Specifications	97
G.1	Model registry record	97
G.2	Cost model	97
G.3	Time model	97
G.4	Progress-state emissions	98
G.5	Changepoint model	98
G.6	Routing bandit	98
G.7	Calibration requirements	98
G.8	Model-card example	98
H	UI Screen and Interaction Specification	100
H.1	Portfolio command center	100
H.2	Project workspace	100
H.3	Live execution event card	100
H.4	Cost explorer	100
H.5	Decision inbox	101
H.6	Sentinel screen	101
H.7	Visual system	101
H.8	Accessibility	101
I	Detailed Repository Tree	102
I.1	Ownership	104
J	Initial Engineering Backlog	105
J.1	M0 - Foundation	105
J.2	M1 - Closed-loop capability spike	105
J.3	M2 - Contract and budget	106
J.4	M3 - Durability and integration	106

J.5	M4 - Multi-user POSM	106
J.6	M5 - Sentinel and portfolio	106
K	Failure-Mode and Recovery Matrix	107
K.1	Recovery principle	108
	References	109

List of Figures

4.1	Reference process topology. Model and credential boundaries are separated from worker execution.	8
20.1	Project lifecycle. State transitions are validated by the domain layer and workflow.	44

List of Tables

Part I

Technical Baseline

Purpose, Scope, and Conformance

1.1 Purpose of this volume

The general system specification explains what qFoundry is and why POSM intends to build it. This document defines the implementable system. It resolves architecture choices, establishes process boundaries, defines schemas and algorithms, and supplies acceptance evidence for each subsystem.

The system is not a chat wrapper. It is a durable control system that coordinates human product owners, an OpenAI supervisor, a Claude coding worker, deterministic policy services, repository and sandbox infrastructure, a cost ledger, and the Sentinel analytics subsystem. The core engineering problem is to preserve human intent while allowing long-running autonomous implementation under hard resource and safety constraints.

1.2 Scope

This volume covers:

- a Tauri desktop application and private remote access;
- a TypeScript control plane and Python statistical service;
- OpenAI and Anthropic model adapters;
- durable workflow orchestration;
- sandboxed worker execution and Git integration;
- append-only event telemetry and live visualization;
- project contracts, contract compilation, milestones, and change control;
- exact internal accounting, conservative reservation, and budget hard stops;
- preflight estimation, scheduling, digital-twin simulation, and Sentinel learning;
- multi-user POSM deployment with administrator oversight;
- security, secrets, retention, backup, recovery, and release engineering.

Out of scope for the first release are automatic production deployment, automatic merge to protected branches, unrestricted browsing, direct model access to long-lived secrets, and autonomous modification of safety policy.

1.3 Conformance classes

An implementation may claim one of four conformance classes.

Class	Required capability
Core	One project, one supervisor, one Claude worker, one sandbox, one budget ledger, and durable pause/resume.
POSM Single Host	Core plus approved-UMN authentication, multiple logical projects, two active workers, master administration, email escalation, and Sentinel advisory mode.
POSM Foundry	Single Host plus dedicated worker nodes, multi-user quotas, portfolio scheduling, digital twin, and Sentinel automatic pauses under bounded authority.
Public Self-Hosted	Foundry capabilities packaged for independent installation with user-supplied provider keys and configurable local policy.

Normative Requirement

Every executable capability shall map to at least one requirement identifier, one test or review method, and one evidence artifact. A feature without traceability is not conformant even if it appears to work manually.

1.4 Implementation philosophy

The design follows five rules.

1. **Deterministic controls dominate model judgment.** Budgets, filesystem boundaries, user authorization, and protected-branch controls are enforced by code.
2. **Models are event driven.** Supervisors and Sentinel models wake on meaningful events instead of continuously consuming tokens.
3. **Evidence outranks narrative.** Completion is established by repository state, tests, artifacts, and requirement evidence, not by a worker's prose claim.
4. **Every side effect is attributable.** The system records the human or model instruction, policy decision, tool call, and resulting state transition.
5. **The first implementation is a modular monolith.** Process isolation is used where security or failure containment requires it; unnecessary microservices are avoided.

Locked Product and Technical Decisions

This chapter converts the design conversation into implementation constraints.

2.1 Product identity and release

The working product name is POSM QFOUNDRY. The software remains private to POSM during the first validation period, then is intended for free public release under a strong copyleft license. Public users supply their own model-provider credentials and control their own policies. The POSM-hosted installation applies institutional budgets, user approval, intake screening, and resource quotas.

2.2 POSM deployment policy

- Only administrator-approved UMN accounts may access the hosted system.
- An ordinary user may have only one active project. The master administrator is exempt.
- Users define and approve their own project contract before submission.
- POSM intake review checks safety, resource fit, and budget before execution.
- The initial maximum funded amount is \$30.00 per project per user.
- Unused value remains in the user's internal account; it is not transferred automatically.
- A user may inspect only projects for which the user has membership.
- The master administrator may inspect all projects and security records.
- Deliberate abuse causes immediate freeze, evidence preservation, and administrator review; confirmed abuse causes permanent suspension.

2.3 Worker and supervisor roles

The OpenAI supervisor owns discovery, contract consistency, task decomposition, worker prompting, routine permission judgment, evidence review, and escalation. Claude is the only primary coding worker in the baseline. Sonnet is used for routine implementation; Opus is reserved for policy-defined high-complexity work. A separate reviewer may be invoked at milestone boundaries.

2.4 Repository operations

Claude may create local commits and, with project-level authorization, push feature branches and open pull requests. Merge to a protected branch requires two explicit user confirmations: acceptance of the

reviewed deliverable and confirmation of the exact merge commit or pull request head.

2.5 Cost semantics

A project budget comprises a target, optional tolerance, and absolute ceiling. The system may stop below the target if the next bounded model call cannot be safely reserved. The live dashboard distinguishes estimated, reserved, pending, and reconciled spend. Dollar values are stored using fixed-point decimal representation.

2.6 Human communication

For POSM, outbound email is allowed from 08:00 through 23:00 America/Chicago, with a default maximum of ten project emails per user per day. Security controls act immediately at any hour, but non-emergency email waits for the next communication window. Reply-by-email is allowed for ordinary project decisions; red permissions and budget increases require secure dashboard confirmation.

2.7 Sentinel authority

Sentinel may pause or quarantine work under predefined cost, security, stagnation, or telemetry-integrity conditions. It may not change contracts, increase budgets, alter safety invariants, access raw credentials, modify immutable telemetry, or merge its own changes.

Architecture Drivers and Quality Attributes

3.1 Priority order

When requirements conflict, implementation decisions shall use the following order:

1. protection of credentials, host data, and other users;
2. enforcement of absolute budget ceilings;
3. preservation of project and audit state;
4. correctness and traceability of delivered work;
5. operator visibility and intervention;
6. throughput and latency;
7. visual polish.

The interface is expected to be visually excellent, but appearance may not obscure uncertainty, estimated values, or failed controls.

3.2 Reliability objectives

The control plane should survive process restarts without losing durable state. A repeated external side effect shall be prevented through idempotency keys or detected and reconciled. Worker crashes may lose in-memory subprocess state, but not the last durable checkpoint, repository commit, budget ledger, or project workflow position.

3.3 Security objectives

No model process receives the raw OpenAI, Anthropic, Gmail, GitHub App, or Tailscale credential. Worker containers receive only project-scoped ephemeral material where unavoidable. The host home directory, other repositories, browser profiles, SSH material, and credential stores are never mounted into worker containers.

3.4 Cost accuracy objectives

Internal accounting shall be exact relative to recorded provider usage and the versioned pricing table. Live values may be estimates until provider usage is returned. Reconciliation lag and uncertainty shall be visible. Absolute ceilings are protected through pre-call reservation and provider-side caps where available. The

Claude Agent SDK exposes per-query budget and result-cost controls, including `maxBudgetUsd`, `maxTurns`, streamed messages, permission callbacks, and interrupt support [5].

3.5 Observability objectives

The operator shall be able to determine, without trusting model prose:

- which requirement is being pursued;
- which prompt was issued;
- which tools and commands were invoked;
- which files changed;
- what tests ran and what failed;
- how much cost was incurred;
- what evidence changed the project's progress state;
- why a policy decision was allowed or denied.

3.6 Performance objectives

On the local network, ordinary dashboard actions should complete within 300 ms at the 95th percentile, excluding model and build operations. New worker events should appear in the desktop stream within two seconds under normal load. Budget checks and permission decisions must complete before the associated external side effect is dispatched.

3.7 Evolvability objectives

Provider APIs, model names, prices, and SDK behavior change. Adapters shall isolate vendor-specific payloads from the domain model. Versioned capability discovery and integration tests are required before enabling a new provider or model in production.

Reference Architecture

4.1 System topology

The baseline uses a modular control plane with explicit process boundaries for security and fault isolation.

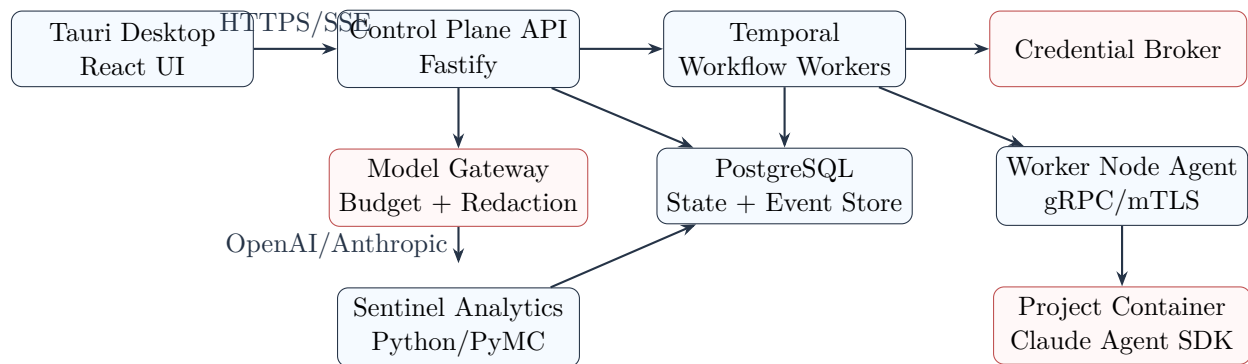


Figure 4.1: Reference process topology. Model and credential boundaries are separated from worker execution.

4.2 Technology baseline

Concern	Baseline	Rationale
Desktop shell	Tauri 2 + Rust shell	Small native package, Windows integration, explicit capabilities.
Frontend	React + TypeScript + Vite	Mature component ecosystem and shared domain types.
Control API	Fastify + TypeScript	Low overhead, schema-first routes, direct streaming support.
Workflow durability	Temporal TypeScript SDK	Durable timers, signals, retries, and human wait states.
Database	PostgreSQL	Transactional ledger, row security, JSONB event payloads, strong tooling.
ORM/query	Drizzle ORM + SQL migrations	Type-safe access while retaining explicit SQL control.

Supervisor	OpenAI Agents SDK	Sessions, streaming, guardrails, traces, and structured outputs [21–23].
Coding worker	Claude Agent SDK Type-Script	Built-in coding tools, sessions, permission callbacks, cost caps, checkpointing, and interruption [4, 5].
Sentinel	Python 3.12, FastAPI/gRPC, PyMC/ArviZ/Polars	Mature Bayesian and statistical diagnostics ecosystem.
Container runtime	Docker Engine rootless on Linux	Familiar isolation, resource controls, and reproducible images [9, 10].
Telemetry	OpenTelemetry + append-only domain events	Standard traces and metrics plus domain-specific provenance [25].
Remote access	Tailscale Serve	Private access without public inbound ports [29].

4.3 Modular-monolith boundary

The initial control plane is deployed as a small set of processes from one monorepo, not dozens of network services. Modules communicate in-process where no security boundary is required. Separate processes are used for the model gateway, credential broker, workflow workers, worker-node agent, and Sentinel because those components have distinct credentials, resource profiles, or failure modes.

Design Principle

Service boundaries are justified by credential isolation, untrusted-code execution, independent scaling, or language/runtime needs. A module is not promoted to a network service merely because it has a different conceptual name.

4.4 Deployment evolution

The same domain and API contracts support three stages:

1. all components on one Windows host with WSL2/Linux containers;
2. Linux control server plus Windows desktop clients;
3. Linux control server plus multiple heterogeneous worker nodes.

The local desktop does not own authoritative project state once the lab server is introduced. It becomes an authenticated client.

Part II

Codebase and Runtime Architecture

Monorepo and Build Architecture

5.1 Repository organization

The qFoundry source shall use a pnpm workspace with Turborepo-compatible task definitions. The Python Sentinel package remains in the same Git repository but uses an isolated Python environment and lockfile.

Listing 5.1: Top-level source layout

```
1  qfoundry/  
2    apps/  
3      desktop/ # Tauri shell + React client  
4      control-api/ # Fastify REST/SSE/WebSocket API  
5      workflow-worker/ # Temporal workflows and activities  
6      model-gateway/ # OpenAI/Anthropic routing and ledger hooks  
7      credential-broker/ # privileged credential proxy  
8      node-agent/ # worker-node daemon  
9      sentinel-service/ # Python statistical service  
10   packages/  
11     domain/ # entities, IDs, state machines  
12     contracts/ # Zod/JSON schemas and API types  
13     policy/ # deterministic permission engine  
14     event-catalog/ # event definitions and serializers  
15     pricing/ # versioned pricing and token accounting  
16     ui/ # shared visual system  
17     testkit/ # fixtures, fake providers, canary repos  
18   infra/  
19     docker/  
20     temporal/  
21     postgres/  
22     tailscale/  
23     packaging/  
24   docs/  
25     adr/  
26     threat-model/  
27     runbooks/
```

5.2 Dependency rules

The dependency graph is one-directional:

`apps → packages/contracts, domain, policy → no app dependency.`

Domain packages shall not import provider SDKs, database clients, Tauri APIs, or HTTP frameworks. Provider payloads are translated at adapter boundaries.

5.3 Build and test commands

The root workspace shall expose stable commands:

```
1  pnpm bootstrap # verify runtimes and install dependencies
2  pnpm dev # desktop + API + local workflow worker
3  pnpm lint # ESLint and formatting checks
4  pnpm typecheck # TypeScript project references
5  pnpm test # unit and package integration tests
6  pnpm test:e2e # isolated end-to-end suite
7  pnpm test:canary # adversarial repository suite
8  pnpm build # reproducible release build
9  pnpm package:windows # signed/unsigned Tauri installer
10 pnpm db:migrate # apply checked migrations
11 pnpm verify:release # complete release gate
```

5.4 Configuration

Configuration is hierarchical and validated at startup:

1. compiled defaults;
2. deployment profile file;
3. encrypted local settings;
4. environment-specific secrets resolved by the broker;
5. per-tenant policy;
6. per-project approved policy reductions.

A lower layer may tighten permissions or budgets but may not widen a global invariant without administrator action.

5.5 Versioning

All externally persisted types carry a schema version. Database migrations are forward-only in production; rollback is performed through restoration and a corrective migration, not by casually reversing a migration that may have transformed data. Event payloads use explicit upcasters for old versions.

Control Plane Services

6.1 Responsibilities

The control plane is the authoritative system of record and command admission point. It owns users, projects, contracts, budgets, permissions, schedules, workflow identifiers, and event access. It does not execute untrusted project commands directly.

6.2 Process decomposition

Process	Responsibilities
control-api	Authentication, authorization, project CRUD, contract interaction, budget views, decisions, event streaming, artifacts, administration.
workflow-worker	Temporal workflow definitions and trusted activities; coordinates supervisor, worker, verification, pauses, and retries.
model-gateway	Provider calls, model catalog, token/cost reservation, redaction, usage capture, and provider-failure normalization.
credential-broker	GitHub, Gmail, and other privileged actions using short-lived or broker-held credentials.
node-agent	Starts/stops containers, streams process output, enforces resource envelopes, creates worktrees, and uploads artifacts.
sentinel-service	Statistical fitting, posterior updates, anomaly scoring, simulation, calibration, and model cards.

6.3 Command handling

Every mutating API request becomes a command with an idempotency key. The API validates identity and syntax, writes a command record, and signals the relevant Temporal workflow or invokes a transactional application service. A request is never considered successful solely because it was accepted into memory.

Listing 6.1: Command envelope

```
1 interface CommandEnvelope<T> {
2     commandId: string;
3     idempotencyKey: string;
4     tenantId: string;
```

```
5   actor: { type: "user" | "service"; id: string };
6   projectId?: string;
7   issuedAt: string;
8   expectedVersion?: number;
9   payload: T;
10 }
```

6.4 Read model

Dashboard queries use denormalized read models maintained transactionally or asynchronously from domain events. The authoritative write state remains normalized. Expensive timeline and analytics queries use materialized views or dedicated summary tables rather than scanning raw events for every screen refresh.

6.5 Consistency

Budget reservations, permission decisions, and project state transitions require strong transactional consistency. Live dashboard projections may be eventually consistent within a two-second objective. The UI displays an “as of” timestamp and event sequence so users can recognize lag.

Desktop Client Architecture

7.1 Application model

The desktop application is a Tauri 2 shell containing a React interface. The shell provides secure local storage for refresh tokens, system tray integration, native notifications, update handling, and optional local-server discovery. Tauri capabilities shall allow only the minimum commands needed by the UI [30, 31].

7.2 Frontend stack

The baseline frontend uses:

- React and TypeScript;
- TanStack Router for typed routes;
- TanStack Query for server state and cache invalidation;
- Zustand for ephemeral local UI state;
- Monaco Editor for code and diff views;
- xterm.js for operator takeover terminals;
- ECharts or Plotly for dense interactive cost, schedule, and posterior visualizations;
- a POSM-specific component system implemented over accessible primitives.

7.3 Data transport

Ordinary reads and commands use HTTPS JSON APIs. Live project events use Server-Sent Events because the primary stream is server-to-client and automatic reconnection is valuable. Interactive terminal takeover uses a separate authenticated WebSocket channel with explicit session creation and inactivity timeout.

7.4 Client state

The UI never infers authoritative project state from visual component state. Each screen binds to versioned server resources. Optimistic updates are permitted only for reversible preferences and draft text. Budget changes, permissions, project transitions, and merge confirmation wait for server acknowledgement.

7.5 Information density

The default mode is engineering-dense. A project screen uses three synchronized panes: navigation, live execution stream, and contextual inspector. Executive and forensic density modes alter presentation but not data availability.

7.6 Emergency controls

Pause, stop-after-command, interrupt-now, quarantine, and global emergency stop remain visible in the top command bar. Destructive controls require deliberate press-and-hold or confirmation, but the global stop must be accessible in at most two actions from any screen.

7.7 Offline and reconnect behavior

If the client loses connectivity, it enters read-only disconnected mode using cached recent state. Commands are not silently queued, except draft text. On reconnect, the client requests events after its last durable sequence and reconciles the view. Terminal takeover is closed immediately on disconnect.

Worker Node and Executor Architecture

8.1 Node agent

Each compute machine runs a small privileged node agent, but the agent itself does not contain model credentials. It accepts authenticated, signed work assignments from the control plane over mutual TLS. The agent creates the project worktree, launches the container, streams events, and enforces resource limits.

8.2 Worker lease

A job is represented by a renewable lease containing project, task, image digest, repository commit, resource envelope, allowed network profile, and expiration. If the node cannot renew the lease, it stops initiating new commands, checkpoints the repository, and terminates after a grace period.

8.3 Container construction

Images are layered:

1. minimal trusted base image;
2. language toolchain image, such as Node, Python, Rust, or FPGA tooling;
3. project dependency cache layer;
4. ephemeral project container with read-write worktree.

Images are referenced by digest. A project may request a custom image, but it must pass policy review and vulnerability scanning.

8.4 Mounted filesystem

The standard container sees only:

```
1 /workspace read-write project worktree
2 /reference optional read-only approved material
3 /qf/tools read-only qFoundry helper binaries
4 /tmp size-limited temporary filesystem
```

The Docker socket, host home, SSH directory, provider credentials, other project volumes, and host device tree are absent.

8.5 Resource envelope

The node agent applies CPU quota, memory hard limit, process count, disk quota, I/O limits, wall-clock timeout, and allowed ports. Out-of-memory termination is reported as a distinct event and never reclassified as an ordinary test failure.

8.6 Command telemetry

Commands are launched through a wrapper that records the exact executable, arguments, working directory, environment-variable names, start time, exit code, resource usage, and redacted output. Shell strings are retained for display, but structured execution is preferred where possible.

8.7 Operator takeover

Takeover creates a new terminal process inside the same container and project namespace. The AI worker is paused before the shell becomes interactive. Commands entered by the user are attributed to that user, not to Claude, and appear in the event stream. Control returns to the supervisor only after the user closes the takeover session and confirms resumption.

Durable Workflow Runtime

9.1 Why Temporal is authoritative for execution state

A qFoundry project may wait hours for a user, survive a server restart, retry an external integration, or pause at a budget threshold. The workflow state therefore cannot live only in a web process or model conversation. Temporal provides durable workflow history, timers, signals, and activity retries suited to human-in-the-loop systems [32, 33].

9.2 Workflow hierarchy

The baseline defines these workflows:

- **ProjectLifecycleWorkflow**: contract approval through user acceptance;
- **MilestoneWorkflow**: execution and verification of one milestone;
- **WorkerTurnWorkflow**: one supervisor instruction and one bounded Claude query;
- **PermissionWorkflow**: resolution of yellow or red permission requests;
- **SalvageWorkflow**: graceful budget/resource shutdown;
- **EmailEscalationWorkflow**: notification, reply correlation, and expiration;
- **SentinelReviewWorkflow**: scheduled or triggered portfolio analysis;
- **ArtifactRetentionWorkflow**: archive and deletion timers.

9.3 Determinism rule

Temporal workflow code must remain deterministic. Network calls, database writes outside workflow state, model requests, Git operations, filesystem actions, and random sampling run as activities. Workflow code stores decisions and coordinates activities; it does not perform side effects directly.

9.4 Signals and queries

Project workflows accept typed signals such as:

```
1 type ProjectSignal =
2   | { type: "USER_DECISION"; decisionId: string; value: unknown }
3   | { type: "PAUSE_REQUESTED"; mode: "graceful" | "immediate" }
4   | { type: "RESUME_REQUESTED" }
5   | { type: "BUDGET_CHANGED"; accountVersion: number }
6   | { type: "PERMISSION_RESOLVED"; requestId: string }
```

```

7 | { type: "EMAIL_REPLY_RECEIVED"; messageId: string }
8 | { type: "SENTINEL_ACTION"; actionId: string };
    
```

Queries expose current phase, active task, waiting reason, last checkpoint, and pending decisions without changing workflow state.

9.5 Idempotent activities

Activities may be retried. Every externally visible activity receives an idempotency key derived from workflow ID, run ID, activity type, and logical operation ID. Examples include email send, Git branch creation, pull-request creation, and artifact upload. An activity first checks the idempotency table, then performs the operation, then stores the provider result transactionally where possible.

9.6 Continue-as-new

Long project workflows periodically continue as new to limit history size. Before continuing, the workflow writes a compact durable snapshot containing contract version, milestone state, session identifiers, unresolved decisions, budget account versions, and last event sequence.

9.7 Cancellation and interruption

Immediate stop signals cancel active activities and invoke the node agent's container interrupt. Graceful pause waits for a safe command boundary, then checkpoints repository state. A cancellation is not marked successful until the node confirms process termination or the node lease expires and Sentinel verifies the process is absent.

Implementation Caution

A Temporal retry does not imply that an external side effect is safe to repeat. Idempotency is an application requirement. Email, Git, and ledger operations must be explicitly deduplicated.

OpenAI Supervisor Runtime

10.1 Supervisor responsibilities

Each project has a logical supervisor identity and session. The supervisor is invoked for discovery, contract compilation review, task planning, worker-result assessment, yellow permissions, change-request analysis, and user escalation. It is not continuously active.

10.2 SDK use

The supervisor uses the OpenAI Agents SDK with a production database-backed session implementation. The SDK supports persistent sessions, streamed runs, resumable interrupted state, tool and output guardrails, and tracing [21, 22, 24]. qFoundry stores its own domain memory separately; SDK session memory is not the authoritative contract database.

10.3 Supervisor tools

The supervisor receives narrow, structured tools:

- `read_contract_sections`;
- `read_current_milestone`;
- `inspect_git_diff`;
- `read_test_report`;
- `read_event_window`;
- `create_worker_instruction`;
- `request_permission_decision`;
- `propose_change_request`;
- `escalate_to_user`;
- `mark_requirement_evidence`;
- `recommend_model_route`.

It does not receive arbitrary host shell access.

10.4 Structured decision output

Every operational supervisor run returns a schema-validated decision.

```

1 interface SupervisorDecision {
2   decision: "CONTINUE" | "CORRECT" | "VERIFY" |
3     "ADVANCE" | "ASK_USER" | "PAUSE" | "COMPLETE";
4   summary: string;
5   evidenceEventIds: string[];
6   requirementIds: string[];
7   workerPrompt?: string;
8   permissionRecommendation?: {
9     requestId: string;
10    outcome: "ALLOW" | "DENY" | "ESCALATE";
11    reason: string;
12  };
13  modelRoute?: "SONNET" | "OPUS";
14  confidenceClass: "LOW" | "MODERATE" | "HIGH";
15  unresolvedRisks: string[];
16 }

```

The `confidenceClass` is a qualitative declaration by the supervisor and is never displayed as a calibrated probability. Statistical probabilities are supplied only by Sentinel models.

10.5 Prompt assembly

The supervisor prompt is generated from immutable policy, project contract extracts, current state, recent evidence, and the specific decision question. Stable prefixes are ordered first to maximize provider prompt caching [20]. The prompt includes event IDs rather than duplicating full logs when a tool can retrieve them.

10.6 Guardrails

Input guardrails reject attempts to override global invariants, expose another tenant's data, or insert untrusted repository text as supervisor authority. Tool guardrails authorize every structured action. Output guardrails validate schema, cited evidence existence, contract version, and allowed state transition. OpenAI tool guardrails can allow, reject, or halt tool execution and retain results for audit [23].

10.7 Discovery mode

Discovery uses a separate supervisor configuration that asks exhaustive product questions and maintains an unresolved-question ledger. It may not create worker tasks. The contract becomes executable only after the user approves a rendered diff and the intake reviewer accepts safety and resource constraints.

10.8 Failure behavior

If the supervisor returns invalid structured output twice, the project pauses with a deterministic error. If the supervisor recommends an action inconsistent with the state machine or policy engine, the action is denied and logged as a supervisor-policy disagreement for Sentinel evaluation.

Claude Coding Worker Adapter

11.1 Adapter boundary

The Claude adapter converts a qFoundry task packet into one bounded Claude Agent SDK query, streams SDK messages into normalized events, resolves tool permissions, and returns a structured worker result. No other subsystem depends directly on Anthropic-specific message types.

11.2 Session lifecycle

Each project uses a persistent Claude session ID. A query may resume the session, but each milestone turn has explicit limits. The Agent SDK supports session persistence, resume, maximum turns, maximum USD budget, partial messages, file checkpointing, custom permission callbacks, and interruption [5].

Listing 11.1: Worker query construction

```
1  const stream = query({
2    prompt: compiledTaskPacket,
3    options: {
4      sessionId,
5      resume: priorSessionId,
6      cwd: "/workspace",
7      model: route.modelId,
8      maxTurns: route.maxTurns,
9      maxBudgetUsd: reservation.providerCapUsd,
10     includePartialMessages: true,
11     enableFileCheckpointing: true,
12     permissionMode: "dontAsk",
13     allowedTools: policy.autoApprovedTools,
14     disallowedTools: policy.deniedTools,
15     canUseTool: permissionCallback,
16     settingSources: ["project"],
17     sandbox: sdkSandboxSettings,
18     outputFormat: { type: "json_schema", schema: workerResultSchema }
19   }
20 });
```

11.3 Why dontAsk

The SDK must never open an unmanaged interactive prompt. Unlisted tool calls route to `canUseTool`; unresolved calls are denied or converted into qFoundry permission requests. The dangerous bypass mode is disabled in all production profiles.

11.4 Message normalization

SDK messages are mapped into qFoundry events:

Claude message	qFoundry event
System initialization	<code>worker.session.initialized</code>
Partial assistant output	<code>worker.message.delta</code>
Assistant message	<code>worker.message.completed</code>
Tool request	<code>worker.tool.requested</code>
Permission denial	<code>permission.denied</code>
Status/compaction	<code>worker.context.status</code>
Result success	<code>worker.turn.completed</code>
Max-budget result	<code>budget.provider_cap_reached</code>
Execution error	<code>worker.turn.failed</code>

11.5 Permission callback

The callback classifies a tool request using the deterministic policy engine. Green requests return allow. Red requests return deny with interruption and create a human escalation. Yellow requests pause the worker turn, persist the request, and wait for a supervisor or human decision. The callback may sanitize or narrow inputs before allowing them.

11.6 Worker result

The worker is required to return:

- concise implementation summary;
- changed files and commits;
- commands and test outcomes;
- requirements believed satisfied;
- unresolved failures and assumptions;
- requested next action;
- explicit statement of whether completion is claimed.

This report is evidence-indexing metadata, not proof. The verification subsystem independently inspects the repository.

11.7 Context management

Claude’s conversation is allowed to compact, but the task packet always reasserts current contract version, milestone definition, safety boundaries, and forbidden changes. If a session becomes polluted or repeatedly

fails, the supervisor may fork a fresh session from a generated handoff packet and the same repository checkpoint.

Contract Compiler

12.1 Compiler objective

The human contract is rich, readable, and versioned. Claude should not receive the entire document on every turn. The compiler produces minimal, task-specific execution packets while preserving traceability and authority.

12.2 Compiler pipeline

1. Parse contract YAML/JSON and validate the schema.
2. Resolve requirement and milestone references.
3. Detect contradictions, undefined terms, cycles, and missing acceptance evidence.
4. Build a requirement dependency graph.
5. Classify requirements by security, architecture, UI, data, and verification relevance.
6. Select the current milestone subgraph.
7. Retrieve applicable decisions and change requests.
8. Build a repository map and relevant symbol index.
9. Render a provider-neutral task intermediate representation.
10. Render model-specific packets for the supervisor, Claude, verifier, and reviewer.
11. Hash the source contract, selected sections, and output packet.

12.3 Intermediate representation

```
1 interface TaskIR {
2   contract: { id: string; version: string; hash: string };
3   projectEssence: string[];
4   task: {
5     id: string;
6     objective: string;
7     inScope: string[];
8     outOfScope: string[];
9     requirementIds: string[];
10  };
11  architectureRules: RuleRef[];
12  securityRules: RuleRef[];
13  acceptanceChecks: CheckSpec[];
14  repositoryContext: RepositorySlice;
```



```

15   knownDecisions: DecisionRef[];
16   allowedTools: string[];
17   responseSchemaVersion: string;
18 }

```

12.4 Contradiction detection

Static checks catch direct contradictions such as mutually exclusive enum values, conflicting permissions, duplicate requirement IDs, and missing dependencies. A supervisor review catches semantic conflicts that require judgment. Every conflict remains open until resolved by the user or explicitly documented as a conditional requirement.

12.5 Readiness score

The compiler reports coverage rather than a single magical score. Required dimensions include behavior completeness, failure-mode completeness, evidence automability, architecture closure, security closure, and unresolved decision count. A milestone is blocked if a must-level requirement lacks a testable completion criterion.

12.6 Packet composition

The Claude packet contains, in order:

1. non-overridable safety and contract precedence;
2. project essence and non-goals;
3. current milestone and exact task;
4. relevant architecture and security decisions;
5. repository map and relevant files;
6. required checks and artifacts;
7. forbidden changes;
8. response schema.

Stable content is placed before dynamic content to improve caching.

12.7 Compiler tests

Golden contracts verify deterministic output. Mutation tests remove requirement fields and assert rejection. Differential tests compare raw-contract and compiled-contract worker performance using missed requirements, rework cost, and acceptance success as outcomes.

Policy and Permission Engine

13.1 Design

The policy engine is deterministic and executes before model judgment. It evaluates an action against global invariants, deployment policy, user role, project policy, task scope, container identity, and current state.

13.2 Policy inputs

```
1 interface PolicyInput {
2     actor: ActorIdentity;
3     tenantId: string;
4     projectId: string;
5     taskId?: string;
6     action: string;
7     resource: ResourceDescriptor;
8     parameters: Record<string, unknown>;
9     state: ProjectStateSnapshot;
10    riskSignals: RiskSignal[];
11 }
12
13 interface PolicyDecision {
14     outcome: "ALLOW" | "DENY" | "SUPERVISOR" | "HUMAN";
15     ruleIds: string[];
16     reasonCode: string;
17     sanitizedParameters?: Record<string, unknown>;
18     expiresAt?: string;
19 }
```

13.3 Evaluation order

1. deny-list and invariant rules;
2. tenant and user authorization;
3. project and task scope;
4. resource and filesystem boundary;
5. command/package/network classifier;
6. state-dependent policy;

7. green auto-allow rules;
8. supervisor-eligible yellow rules;
9. human-only red rules.

A denial at a higher priority cannot be overridden by a lower-priority allow.

13.4 Command classification

Shell commands are parsed into an executable and argument vector when possible. The engine maintains command families: read-only inspection, local build/test, package manager, Git local, Git remote, network, process control, filesystem deletion, system administration, and unknown. Compound shell syntax, redirection, command substitution, or encoded payloads raise the classification risk.

13.5 Dependency approval

A dependency request includes registry, canonical package identity, version, lockfile delta, license, vulnerability status, install scripts, native code, maintenance signals, and stated reason. Low-risk common packages may be auto-approved and reported to the supervisor. Native binaries, abandoned packages, suspicious name similarity, new registries, or broad postinstall behavior require escalation.

13.6 Policy language

Initial rules are implemented in typed TypeScript and versioned YAML data, not a general-purpose policy language. Once the rule set stabilizes, Open Policy Agent may be evaluated. The v0.1 priority is debuggability and exact tests rather than introducing another runtime.

13.7 Explainability

Every permission event shows the matched rule IDs, facts used, sanitized inputs, decision authority, and whether a human or model participated. “Allowed by AI” is never an acceptable reason.

Model Gateway

14.1 Purpose

All OpenAI and Anthropic traffic in the POSM deployment passes through the qFoundry model gateway. The gateway centralizes provider credentials, capability discovery, request policy, budgeting, rate limits, redaction, model routing, caching metadata, and usage normalization. Anthropic explicitly supports LLM-gateway deployment for centralized routing and control [6].

14.2 Provider abstraction

```

1 interface ModelProviderAdapter {
2     listCapabilities(): Promise<ModelCapability[]>;
3     estimate(request: CanonicalModelRequest): Promise<CostBound>;
4     start(request: CanonicalModelRequest,
5           reservation: BudgetReservation): AsyncIterable<ModelEvent>;
6     interrupt(executionId: string): Promise<void>;
7     reconcile(providerUsage: unknown): Promise<NormalizedUsage>;
8 }

```

Provider-specific SDKs may run in dedicated adapter processes, but all requests must present a valid qFoundry reservation token.

14.3 Model catalog

The model catalog stores aliases instead of embedding volatile provider names in contracts:

```

1 aliases:
2   supervisor_max_reasoning:
3     provider: openai
4     model_id: <configured-current-model>
5   supervisor_economy:
6     provider: openai
7     model_id: <configured-cheaper-model>
8   worker_sonnet:
9     provider: anthropic
10    model_id: <configured-sonnet-model>
11   worker_opus:
12     provider: anthropic
13    model_id: <configured-opus-model>

```

Catalog changes require integration tests and administrator activation.

14.4 Request admission

Admission performs:

1. authentication and project-state validation;
2. model-route authorization;
3. prompt redaction scan;
4. token estimation;
5. worst-case cost bound;
6. budget reservation;
7. concurrency and rate-limit admission;
8. provider dispatch.

If any step fails, no provider request is sent.

14.5 Redaction

Secrets are redacted before tracing and before provider dispatch when the secret is not intentionally required. Redaction uses structured field rules, secret-manager markers, entropy/pattern detectors, and project-specific known-secret hashes. The original raw value is never stored in the event payload.

14.6 Usage normalization

Normalized usage includes input, output, cached input, cache creation, reasoning if billed separately, tool charges, provider request ID, model ID, pricing-table version, estimated cost, and reconciled cost. The raw provider usage is encrypted and retained for a shorter audit period.

14.7 Fallbacks

A provider failure may trigger an approved fallback only when the task contract permits it and semantic behavior is equivalent enough. Coding worker fallback from Sonnet to Opus is a routing escalation, not an outage fallback. Supervisor provider substitution is disabled by default because the product intentionally relies on OpenAI judgment.

Budget Ledger and Hard-Stop Enforcement

15.1 Accounting model

The ledger uses double-entry accounting with fixed-point `NUMERIC(20,8)` dollar amounts. Each user wallet and project subaccount has available, reserved, pending, reconciled, and expired values. Model calls do not directly mutate a “spent” field; they post ledger entries.

15.2 Account hierarchy

```

1 POSM provider funding pool
2   -> user wallet
3     -> project account
4       -> supervisor subaccount
5       -> Claude worker subaccount
6       -> reviewer subaccount
7       -> Sentinel allocation
8       -> salvage reserve

```

The public self-hosted edition may collapse these into one local account while retaining the same ledger code.

15.3 Reservation formula

For a model call with known input tokens n_{in} , maximum output n_{out}^{max} , provider/tool fees f , and uncertainty buffer b , the reserved amount is

$$R = p_{in}n_{in} + p_{out}n_{out}^{max} + f + b,$$

using the active pricing-table version. For a Claude Agent SDK query, qFoundry additionally sets `maxBudgetUsd` to the smaller of the provider-level task cap and the safely available project amount [5].

15.4 Admission inequality

A new call is admitted only if

$$S_{reconciled} + S_{pending} + R + R_{salvage} \leq C_{absolute},$$

where $C_{absolute}$ is the project or subaccount hard ceiling. Reservations are atomic and serialized per account.

15.5 Target and tolerance

A user may configure:

```
1 {
2   "targetUsd": "20.00",
3   "positiveToleranceUsd": "0.50",
4   "absoluteCeilingUsd": "20.50"
5 }
```

The scheduler optimizes toward the target. The ledger enforces the absolute ceiling. The system may stop below the target because no further bounded call fits.

15.6 Reconciliation

After a call completes, the gateway posts actual normalized usage, releases unused reservation, and marks the result estimated or provider-authoritative. Periodic provider-account reconciliation compares aggregate qFoundry records with provider usage reports. Differences beyond tolerance create a financial-integrity incident and freeze new spending if the discrepancy threatens hard-cap guarantees.

15.7 Threshold actions

Thresholds are policy-driven. The POSM default is cost optimization at 70 percent, no unquoted major task at 80 percent, completion forecast at 88 percent, bounded work only at 92 percent, salvage at 95 percent, and hard denial at 100 percent.

15.8 Salvage funding

The salvage reserve is a protected subaccount. Ordinary feature work cannot consume it. It funds only checkpointing, minimal verification, summary generation, session handoff, and one notification. If no provider call is affordable, deterministic salvage still saves repository and ledger state.

Security Invariant

No UI, supervisor, worker, or Sentinel action may directly edit an account balance. All changes occur through validated ledger transactions with immutable entries and idempotency keys.

Preflight Estimation and Quoting

16.1 Purpose

Before a meaningful task begins, qFoundry estimates cost, elapsed time, success probability, deadline probability, uncertainty, resource requirements, and likely human decisions. The quote is a calibrated forecast, not a guarantee.

16.2 Feature extraction

Features include repository language mix, file count, dependency graph, test health, change surface, task category, requirement count, UI or numerical evidence needs, worker model, supervisor model, prior similar tasks, repository maturity, and unresolved uncertainty.

16.3 Cold-start estimation

Before enough local data exists, the estimator uses conservative expert priors and broad intervals. Quotes explicitly state low sample support. Point estimates with false fractional precision are forbidden.

16.4 Candidate plans

Each quote produces at least four candidate plans:

- cheapest feasible;
- recommended balanced;
- fastest reasonable;
- highest-confidence.

Plans differ in task ordering, concurrency, model route, review depth, proof tasks, and scheduling of batchable work.

16.5 Quote object

The quote stores p10, p50, and p90 cost and duration, deadline probability, contract-success probability, resource envelope, assumptions, risk register, critical path, and the evidence that would materially change the estimate.

16.6 Proof-before-build

When uncertainty dominates expected cost, the planner proposes a bounded proof task. The value of information is estimated by comparing expected downstream decision quality with and without the proof. A proof is favored when a small experiment can prevent a large likely waste.

16.7 Calibration

Forecasts are evaluated with empirical coverage, Brier score for binary events, log score, interval width, and calibration plots. Bayesian workflow requires prior predictive checks, posterior predictive checks, sensitivity analysis, and out-of-sample evaluation [8, 12, 34].

16.8 Quote expiry

Quotes expire when the repository changes materially, model pricing changes, contract version changes, a dependency completes, or the scheduled start window passes. Starting a task against an expired quote requires automatic requoting.

Portfolio Scheduler and Digital Twin

17.1 Scheduling problem

The scheduler selects which tasks run, on which node, at what time, with which model and verification plan, subject to budgets, deadlines, user fairness, machine resources, provider rate limits, communication windows, dependencies, and salvage reserves.

17.2 Hard constraints

Hard constraints include one active ordinary-user project, maximum active workers, per-user memory/CPU/disk limits, project and daily budget ceilings, model-specific caps, dependency precedence, worker-node compatibility, email windows, and safety policy.

17.3 Objective

The planner minimizes expected monetary cost, deadline loss, failure/rework loss, human-attention burden, security exposure, and wasted compute while penalizing tail risk using conditional value at risk:

$$\min_{\pi} \mathbb{E}[C_{\pi} + L_D + L_F + L_H + L_S + L_W] + \beta \text{CVaR}_{\alpha}(C_{\pi} + L_D).$$

Weights are deployment policy and are visible to the administrator.

17.4 Two-stage scheduling

A deterministic admission layer first filters infeasible actions. A stochastic planner then compares feasible plans. The scheduler never relies on an LLM to solve arithmetic constraints.

17.5 Fairness

Ordinary users receive weighted fair access. A user with one long project may not monopolize all workers. The master can override priority, but overrides are recorded. Users cannot submit a second project until the first reaches user-accepted, cancelled, rejected, or archived state.

17.6 Digital twin simulation

The digital twin samples task durations, costs, failure/rework events, user response delays, resource contention, and provider throttling. It estimates probabilities of portfolio outcomes and the marginal value of budget, deadline, or priority changes.

17.7 Simulation implementation

V0.1 uses Monte Carlo simulation with explicit distributions supplied by Sentinel. Simulations are seeded and reproducible. Results include confidence intervals and sensitivity rankings. A scenario record stores model versions, posterior snapshot IDs, random seed, and policy assumptions.

17.8 Schedule repair

The scheduler replans after task completion, estimate update, user decision, failure, node outage, budget change, or deadline change. Started tasks are not preempted unless the expected value of preemption exceeds checkpoint cost or a hard constraint requires it.

17.9 User interface

The schedule view shows a Gantt-like timeline, worker/node assignments, cost reservations, critical path, idle causes, and alternative plans. Users may move deadlines or priorities; the server returns a new simulated plan rather than pretending a drag action is automatically feasible.

Sentinel Statistical and Improvement System

18.1 Separation of telemetry and inference

Sentinel consumes immutable validated telemetry. It may create derived features, posterior distributions, predictions, and recommendations. It cannot alter source events. Every prediction references the event-sequence range and feature version used.

18.2 Service architecture

The Python service contains:

- feature materialization jobs using Polars and PostgreSQL;
- Bayesian model fitting using PyMC or an equivalent audited framework;
- posterior storage and ArviZ diagnostics;
- online anomaly and changepoint services;
- simulation endpoints;
- calibration and model-card generation;
- experiment registry and sequential testing.

The TypeScript control plane invokes Sentinel through a versioned gRPC or HTTP schema. Statistical computation does not hold operational credentials.

18.3 Initial model portfolio

18.3.1 Hierarchical cost model

For task i in repository r and category k :

$$\log C_i \sim \mathcal{N}(\mu_i, \sigma_k), \quad \mu_i = \alpha + a_r + b_k + \mathbf{x}_i^\top \boldsymbol{\beta},$$

with partial-pooling priors for repository and task effects. Zero- or near-zero deterministic tasks are modeled separately.

18.3.2 Completion-time model

A survival or accelerated-failure-time model predicts time to technical readiness while handling censored paused tasks. Human wait time is modeled as a separate component so engineering speed is not confused with overnight approval delay.

18.3.3 Progress-state model

A hidden semi-Markov model infers states such as exploring, implementing, verifying, progressing, reworking, stalled, blocked, and complete from event emissions and dwell times. The model is advisory until validated.

18.3.4 Changepoint detection

Bayesian Online Changepoint Detection tracks abrupt shifts in cost rate, error rate, tool pattern, and progress velocity [1].

18.3.5 Routing model

Contextual bandits with knapsack constraints evaluate model choices under money, time, and attention budgets [2]. New routing policies begin in shadow mode and may not violate deterministic route restrictions.

18.3.6 Safe tuning

Safe Bayesian optimization is considered for review frequency, context-pack size, and concurrency, but only within a predefined safe parameter set [11, 28].

18.4 No hand-wavy probabilities

A displayed probability must identify outcome, horizon, model version, sample support, posterior interval, calibration status, and known limitations. LLM judgments are not converted into numeric probabilities.

18.5 Model lifecycle

1. define outcome and decision use;
2. register features and priors;
3. run prior predictive checks;
4. fit on training data;
5. validate on chronological holdout;
6. assess calibration and failure subgroups;
7. historical replay;
8. shadow deployment;
9. bounded trial;
10. administrator activation;
11. continuous drift monitoring.

18.6 Improvement experiments

Sentinel may propose a bounded experiment with hypothesis, intervention, control, maximum spend, stopping rule, success metric, and rollback. Sequential tests stop when evidence is sufficient or the budget

is exhausted. Improvements are delivered by pull request and cannot self-deploy.

18.7 Security monitoring

Sentinel analyzes unusual filesystem denials, dependency requests, network destinations, secret detections, permission patterns, telemetry gaps, and cross-project access attempts. Deterministic detectors trigger immediate containment; statistical models prioritize investigation and reduce false alarms.

Progress and Evidence Engine

19.1 Problem statement

Activity is not progress. Commands, messages, and file modifications may represent exploration, rework, or regression. qFoundry therefore computes progress from requirement evidence and verified state changes.

19.2 Evidence graph

Each requirement links to one or more evidence claims. Claims link to artifacts such as test results, screenshots, build outputs, static-analysis reports, numerical reference comparisons, code review findings, or user acceptance.

```
1 interface EvidenceClaim {  
2   id: string;  
3   requirementId: string;  
4   kind: "TEST" | "BUILD" | "SCREENSHOT" | "REPORT" |  
5       "REVIEW" | "MANUAL_DEMO" | "USER_ACCEPTANCE";  
6   artifactIds: string[];  
7   producedBy: ActorRef;  
8   verifiedBy?: ActorRef;  
9   status: "PROPOSED" | "VALID" | "INVALID" | "STALE";  
10  contractVersion: string;  
11  sourceCommit: string;  
12 }
```

19.3 Progress dimensions

The dashboard displays:

- implemented scope: requirements with implementation evidence;
- independently verified scope: requirements with valid independent evidence;
- milestone health: checks passed, failed, pending, or blocked;
- progress velocity: newly verified weighted scope per dollar and per active hour;
- rework: evidence invalidated or commits reverted;
- time since meaningful progress.

Weights come from contract priority and effort class, not lines of code.

19.4 Evidence staleness

A code change can invalidate earlier evidence. The dependency graph marks claims stale when their source commit is no longer an ancestor, relevant files change, environment version changes, or the acceptance criterion changes. The verifier reruns affected checks before completion.

19.5 Meaningful progress event

A `project.progress.meaningful` event is emitted only when at least one of these occurs:

- a requirement gains valid evidence;
- a blocker is removed;
- a failed acceptance check becomes passing;
- uncertainty is reduced through an approved proof task;
- a critical dependency completes;
- a security or reliability defect is resolved and verified.

19.6 Stagnation detection

Deterministic heuristics first detect repeated identical failures, edit-revert cycles, unchanged test outcomes, or long activity without evidence changes. Sentinel then estimates whether the state is ordinary exploration or likely stagnation. Automatic pause thresholds are conservative and visible.

19.7 Completion

A milestone reaches technical readiness only when all must requirements have valid evidence, all required checks pass, no blocking security finding remains, and the supervisor's evidence review succeeds. User acceptance is a distinct later transition.

Part III

Data, Interfaces, and Integrations

Domain Model and Persistence

20.1 Aggregate boundaries

The primary aggregates are Tenant, User, Wallet, Project, Contract, Milestone, Task, Workflow, WorkerSession, BudgetAccount, PermissionRequest, Decision, Artifact, WorkerNode, and SecurityIncident. Aggregate updates use optimistic concurrency through a version number and transactionally append domain events.

20.2 Identifier policy

Externally visible identifiers use UUIDv7 or time-sortable equivalent. Human-readable codes such as REQ-UI-014 are unique within a contract version but are not database primary keys. Provider IDs are stored as external references and never used as internal identity.

20.3 Project state machine

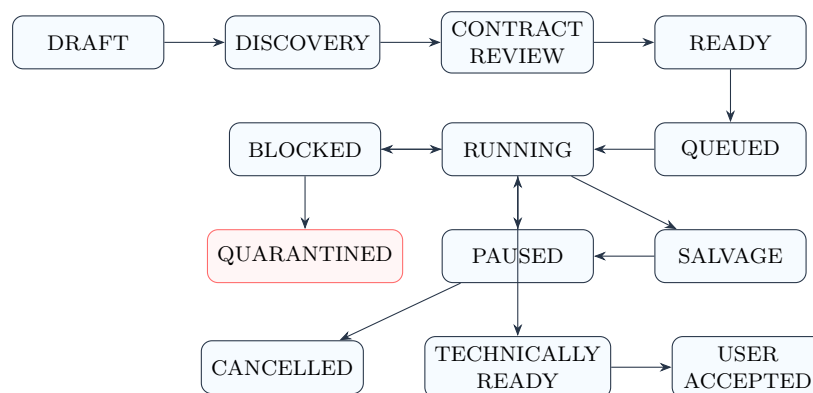


Figure 20.1: Project lifecycle. State transitions are validated by the domain layer and workflow.

20.4 Contract versioning

Contracts are immutable after approval. A change request creates a new draft version derived from the current approved version. Work continues against one active version until the new version is approved and its migration plan accepted. Evidence references the contract version under which it was produced.

20.5 Multi-tenancy

Every tenant-owned row carries `tenant_id`; every project-owned row carries `project_id`. PostgreSQL row-level security provides defense in depth, but application authorization remains mandatory. PostgreSQL row security can restrict visible and modifiable rows according to policy [27].

20.6 Transactional event append

A domain state change and its corresponding event are written in one PostgreSQL transaction. An outbox row is also written for asynchronous projections. A dispatcher publishes the event and marks the outbox item delivered. Consumers are idempotent by event ID.

20.7 Retention classes

Tables distinguish operational state, immutable audit, raw telemetry, artifacts, and derived analytics. Retention jobs may delete raw content according to policy, but they preserve tombstones, cost totals, security incidents, contract decisions, and hashes needed to maintain audit continuity.

Event Store, Telemetry, and Provenance

21.1 Two telemetry layers

qFoundry uses OpenTelemetry for technical traces, metrics, and logs, and a domain event store for business/audit semantics. OpenTelemetry spans are excellent for performance and causality, but domain events remain the authoritative project history [7, 24, 25].

21.2 Event envelope

Every event contains event ID, tenant, project, workflow, session, trace, parent event, actor, event type, schema version, severity, payload, cost delta, security classification, source, timestamps, and provenance hash.

21.3 Append-only rule

Events are never updated in place. A correction emits a compensating event that references the original. Event payloads containing sensitive values are redacted before insertion. Hash chaining is used to detect accidental or malicious modification; it is not presented as a substitute for database access controls and backup integrity.

21.4 Sequence assignment

Each project receives a monotonic event sequence allocated transactionally. The global event ID remains unique across the deployment. Desktop reconnection uses the last seen project sequence.

21.5 Event severity and classification

Severity is debug, info, notice, warning, error, critical. Classification is public, internal, sensitive, or restricted. Restricted events are visible only to the master/security role and may expose redacted security context unavailable to project users.

21.6 Live delivery

The API reads committed events from a projection table and delivers them over SSE. Events are not streamed directly from in-memory worker callbacks before commit. This prevents the UI from showing actions that disappear after a server crash.

21.7 Raw tool output

Long terminal output is stored as a compressed artifact, while the event contains a bounded preview, hash, byte count, encoding, and artifact ID. Secret scanning and redaction occur before the artifact becomes available to users.

21.8 Metrics

Key metrics include model cost, reservation utilization, event lag, permission latency, worker active time, meaningful-progress rate, test pass rate, container resource use, node health, email queue depth, workflow failures, and estimate calibration.

21.9 Trace correlation

Each model call, supervisor run, worker query, command, and verification job shares a trace ID. The UI may open a trace waterfall, but it explains that traces are operational telemetry while the project timeline is the durable record.

API and Inter-Process Contracts

22.1 External API style

The desktop client uses versioned REST endpoints for commands and queries, SSE for event streaming, and WebSocket for interactive terminals. OpenAPI is generated from Zod schemas and checked into the repository for review.

22.2 Core routes

Route	Purpose
POST /v1/projects	Create project draft from idea or connected repository.
POST /v1/projects/:id/discovery/messages	Connect to supervisor discovery conversation.
POST /v1/projects/:id/contracts/apply/approve	Apply and approve contract hash.
POST /v1/projects/:id/intake/submit	Request POSM safety/resource review.
POST /v1/projects/:id/quotes	Generate or refresh preflight quote.
POST /v1/projects/:id/run	Admit approved project to queue.
POST /v1/projects/:id/pause	Graceful or immediate pause.
POST /v1/projects/:id/resume	Resume from durable state.
POST /v1/permissions/:id/resolve	Resolve authorized permission request.
POST /v1/decisions/:id/respond	Submit project decision.
GET /v1/projects/:id/events	SSE stream with after-sequence cursor.
GET /v1/projects/:id/artifacts	List downloadable deliverables/evidence.
POST /v1/projects/:id/merge/prepare	Prepare exact merge confirmation.
POST /v1/projects/:id/merge/confirm	Second-step merge authorization.

22.3 Error model

Errors use machine-readable code, human message, correlation ID, retryability, and optional field violations. Security denials do not reveal hidden resource existence. Budget errors include required reservation, safely available amount, and salvage state.

22.4 Idempotency

All mutating routes require an **Idempotency-Key**. Repeating a request with the same key and same body returns the original result. Reusing a key with a different body returns a conflict.

22.5 Node protocol

Control-to-node communication uses gRPC over mTLS. Commands include provision workspace, start container, run worker turn, interrupt, checkpoint, collect artifacts, and destroy workspace. The node streams signed status and event batches. Nodes cannot self-assign projects.

22.6 Sentinel protocol

Sentinel endpoints are read-only with respect to operational state. They return predictions, anomaly findings, model cards, simulations, and recommendations. Operational actions are translated into control-plane commands and re-evaluated by policy.

22.7 Schema compatibility

Clients send a supported API version and receive a server capability document. Additive fields are allowed within a major version. Removing fields or changing semantics requires a new version and migration window.

Git, Repository, and Artifact Subsystems

23.1 Repository modes

A project begins from either a new repository workspace or a connected existing repository. The GitHub integration uses a GitHub App installed only on selected repositories, with short-lived installation tokens generated by the credential broker [13, 14].

23.2 Worktree strategy

Each active task receives a dedicated branch and worktree. The naming convention includes project and task IDs. Worker containers do not share a writable worktree. Local commits occur at checkpoint boundaries and before high-risk changes.

23.3 Protected operations

Push and pull-request creation require project-level opt-in. Force push, protected-branch updates, repository deletion, release publication, secret modification, and merge are denied or human-only. Merge preparation records exact base and head SHAs; the second confirmation is invalidated if either changes.

23.4 Git event capture

The node agent records repository status before and after a worker turn, changed paths, insertions/deletions, commits, branch, base SHA, and untracked files. Binary artifacts are identified separately so large generated outputs are not accidentally committed.

23.5 Artifact service

Artifacts include build packages, reports, screenshots, terminal logs, test reports, model cards, contract exports, and project archives. Artifacts are content-addressed, checksum verified, classified, and linked to events and requirements.

23.6 Deliverable bundle

User acceptance produces a downloadable bundle containing:

- final contract and approved changes;
- repository or branch reference;
- build artifact where applicable;
- test and verification evidence;
- known limitations;
- reproducibility instructions;
- cost report;
- security and dependency report;
- recommended future work.

23.7 Repository deletion

Deleting a project workspace is a master-admin operation after retention checks. Connected upstream repositories are never deleted by qFoundry. Local clones, worktrees, caches, and artifacts are removed according to policy, with an audit tombstone retained.

Email and Notification Integration

24.1 Dedicated identity

The POSM deployment uses a dedicated qFoundry mailbox. Supervisor names appear as display labels, but messages originate from the qFoundry service rather than impersonating OpenAI or Anthropic.

24.2 Outbound message object

Every notification has severity, project, recipient, category, subject template, body template, decision link, communication window, deduplication key, expiration, and idempotency key. The notification scheduler enforces per-user daily caps and bundles related nonurgent questions.

24.3 Gmail integration

The email service uses Gmail OAuth and Gmail API threads. Gmail push notifications deliver mailbox-change hints through Pub/Sub; the service then retrieves history changes and correlates replies [15–17].

24.4 Reply correlation

An escalation email includes a signed opaque token associated with the project, decision, authorized recipient, and expiration. The service verifies sender allowlist, thread identity, token, and replay status before creating a decision response.

24.5 Interpretation and confirmation

Ordinary free-text replies are interpreted by the project supervisor and shown as a proposed structured decision. Low-risk decisions may resume automatically if policy allows. Budget increases, merge actions, and red permissions require a secure dashboard confirmation of the interpreted action.

24.6 Communication windows

For POSM, nonemergency outbound messages are sent between 08:00 and 23:00 America/Chicago. Messages generated outside the window are queued. Security containment occurs immediately, and a notification is

queued unless the administrator has configured an emergency channel.

24.7 Desktop notifications

The Tauri client displays native notifications for pending decisions, budget thresholds, security events, milestone readiness, and system health. Notification content follows the same redaction classification as email.

Part IV

Security and Multi-Tenancy

Identity, Authorization, and Tenant Isolation

25.1 Authentication

The POSM deployment uses Google OIDC restricted to `umn.edu` identities plus an explicit approved-user table. Domain claims alone are insufficient; an account remains inactive until approved by an administrator. Public self-hosted deployments may configure other OIDC providers or local administrator-created accounts.

25.2 Roles

Roles are master administrator, security administrator, faculty/reviewer, project owner, project contributor, and observer. Role grants are tenant scoped. Project memberships further restrict project access.

25.3 Authorization model

Authorization evaluates role, tenant, project membership, project state, action sensitivity, and resource ownership. The control API checks authorization before loading sensitive data. PostgreSQL row-level security provides a second barrier [27].

25.4 Master account

The master administrator can view all projects, budgets, telemetry, user quotas, and Sentinel findings. Sensitive raw content remains subject to explicit audit logging. Master privileges do not bypass immutable safety invariants or budget accounting.

25.5 User project rule

An ordinary user cannot submit a second project while one is active. Active includes discovery through technically ready unless the project is rejected, cancelled, archived, or user accepted. Drafts may be limited separately to prevent abuse.

25.6 Resource quotas

Each user has maximum active projects, RAM, CPU, disk, process count, runtime, daily spend, project spend, and network profile. The scheduler admits work only when both project and user quotas fit.

25.7 Immediate abuse response

A credible deliberate attempt to access another project, exfiltrate secrets, bypass sandboxing, create malware, or evade policy freezes the account and all active containers. Evidence is preserved. The administrator decides permanent ban and external reporting. The system does not automatically erase records needed for investigation.

25.8 Audit

Authentication, role changes, project membership, data exports, privileged views, and admin decisions are audited. Users can see access to their project records except when an active security investigation requires restricted handling.

Secrets and Credential Broker

26.1 Trust boundary

Provider keys and OAuth refresh tokens are never present in worker-container environment variables, prompts, logs, screenshots, or repository files. The gateway and broker run under distinct service identities with only the credentials they require.

26.2 Storage

On Windows development hosts, the root wrapping secret is protected by the operating-system credential facility and used to unlock an encrypted qFoundry vault. On the Linux lab server, the preferred production design uses an encrypted secret store with host-bound key protection and offline recovery material held by administrators.

26.3 Brokered operations

Claude asks to perform an operation, not to receive a credential. Examples are create branch, open pull request, send email, retrieve approved API data, or upload an artifact. The broker authorizes the structured request, obtains a short-lived credential if available, performs the call, and returns a minimized result.

26.4 Ephemeral project secrets

If project code must use a third-party API during tests, the owner registers a project-scoped secret. The secret is mounted through a temporary file or local proxy only for the approved command, then removed. The model sees the secret name and purpose but not value.

26.5 Redaction pipeline

Redaction runs at command output capture, model prompt construction, event serialization, trace export, artifact upload, email rendering, and UI display. Known secret values are represented by keyed hashes so accidental appearance can be detected without storing cleartext in the scanner.

26.6 Leak response

A suspected secret exposure triggers worker interruption, output quarantine, commit blocking, credential revocation or rotation, repository-history scan, incident creation, and administrator notification. Work resumes only after the security checklist is complete.

26.7 Backups

Vault backups are encrypted separately from database backups. Recovery keys are not stored on the same machine. Restoring a database without the vault yields an operable read-only audit system but cannot make provider calls.

Security Invariant

The model gateway and credential broker must be separate operating-system processes from worker containers. Sharing the host environment with Claude defeats the credential boundary even if prompts instruct Claude not to read keys.

Sandbox, Network, and Dependency Security

27.1 Isolation layers

Security does not rely on one mechanism. The baseline uses an unprivileged container user, rootless container runtime, minimal mounts, read-only root filesystem where practical, seccomp/AppArmor or equivalent profile, dropped capabilities, process and memory limits, network egress proxy, and project-specific credentials.

27.2 Network default deny

Worker containers have no unrestricted outbound network. DNS and HTTP(S) traffic pass through a policy proxy. Approved categories include provider-independent official documentation, selected package registries, project-approved APIs, and Git operations through the broker. Private LAN ranges and cloud metadata endpoints are denied.

27.3 Domain grants

A grant records domain, ports, methods, project, task, reason, approver, and expiration. Wildcards are discouraged. Redirect destinations are revalidated. Raw IP access is denied unless explicitly required.

27.4 Package installation

Package managers are configured to approved registries and lockfiles. The dependency service resolves canonical identity, detects typosquatting indicators, checks license and vulnerabilities, records install scripts/native components, and produces a dependency diff before approval.

27.5 Prompt injection from repository content

README files, issue text, comments, package metadata, fetched documentation, and tool output are untrusted data. The contract packet states that repository content cannot grant permissions or change authority. Suspicious instruction-like content is tagged in the event stream and may trigger a Sentinel review.

27.6 Canary repository

The release suite includes adversarial repositories containing fake secrets, misleading instructions, dangerous scripts, dependency-confusion bait, unsafe symlinks, and attempts to access host resources. Each release must demonstrate containment.

27.7 Windows development versus Linux production

V0.1 may use Docker Desktop/WSL2 for development. The dedicated lab server should run native Linux for simpler service operation and cleaner container isolation. Worker images remain portable across both profiles.

Security Operations and Incident Response

28.1 Threat model maintenance

The threat model covers malicious users, compromised repositories, prompt injection, vulnerable dependencies, model mistakes, credential leakage, cross-project access, budget abuse, telemetry tampering, provider compromise, and physical server access. It is updated for every new privileged integration.

28.2 Incident classes

Class	Examples	Automatic response
SEV-1	Confirmed credential leak, sandbox escape, cross-tenant exfiltration	Stop affected and adjacent workers, revoke credentials, quarantine, alert master.
SEV-2	Suspected malicious dependency, telemetry integrity failure, repeated policy bypass	Pause project, preserve state, require security review.
SEV-3	Denied suspicious command, anomalous network request, repeated unknown package	Continue only if deterministic controls contained it; create review item.
SEV-4	Benign misconfiguration or failed login	Log and expose to user/admin as appropriate.

28.3 Evidence preservation

Incident evidence includes event sequence, trace IDs, redacted outputs, container image digest, repository SHA, process list, policy decision, node identity, and involved credentials. Evidence storage is write-restricted and retained longer than ordinary raw logs.

28.4 Security release gates

Each release runs static analysis, dependency review, secret scanning, container image scanning, authorization tests, canary-repository tests, backup restore test, and policy regression tests. The development process should align with secure software development practices such as NIST SSDF [19] and address agent-specific risks including prompt injection and excessive agency [26].

28.5 Physical controls

The dedicated lab machine resides in an approved locked enclosure with documented power, cooling, and network access. At least one backup authorized administrator should be designated for emergency power-off and recovery even if daily access is restricted to the primary operator.

28.6 Vulnerability response

Public disclosure later requires a security policy, supported versions, private reporting address, severity triage, and release process. Security fixes may bypass ordinary feature scheduling but still require tests and signed release artifacts.

Part V

Deployment, Reliability, and Verification

Deployment and Installation

29.1 Profiles

qFoundry supports development, POSM single-host, POSM lab-server, and public self-hosted profiles. Each profile is a signed configuration bundle defining enabled services, authentication, resource limits, model aliases, email behavior, and network policy.

29.2 Development profile

The developer runs the Tauri client and control processes locally, with PostgreSQL and Temporal in containers. One disposable worker container uses fake or tiny-budget provider credentials. The profile enables verbose logs and fake provider adapters.

29.3 POSM single-host profile

The Windows desktop app connects to services running locally or through WSL2. The host may run up to two active workers. Local remote access is provided through Tailscale, not a public listening port.

29.4 POSM lab-server profile

A dedicated Linux machine runs PostgreSQL, Temporal, control services, gateway, broker, Sentinel, and one or more node agents. Windows desktop applications authenticate remotely. Worker nodes may be the same machine or additional donated x86 workstations.

29.5 Installer behavior

The Windows installer performs prerequisite checks, installs the desktop client, optionally installs a local service bundle, and guides the administrator through provider credentials, OIDC, Gmail, GitHub App, Tailscale, budgets, and backup path. It never writes secrets into a repository-visible configuration file.

29.6 Configuration validation

Startup fails closed when pricing is missing, database migrations are incomplete, the credential vault cannot unlock, a required provider alias is unresolved, or the event hash chain is inconsistent. Noncritical

integrations such as email may be disabled with a prominent degraded-status indicator.

29.7 Upgrades

Upgrades follow prepare, drain, backup, migrate, start, verify, and rollback-decision phases. Active projects are gracefully paused before incompatible workflow or schema changes. Temporal workflow versioning is used when code changes would otherwise alter deterministic replay.

29.8 Public packaging

The public release includes container manifests, Tauri client, setup wizard, sample policies, fake provider/test mode, and documentation for user-supplied keys. POSM-specific caps are defaults in the POSM deployment bundle, not hardcoded restrictions in the engine.

Reliability, Backup, and Recovery

30.1 Failure domains

The design treats desktop client, control API, Temporal worker, PostgreSQL, model provider, node agent, worker container, email integration, and Sentinel as separate failure domains. A failure in one must not corrupt authoritative state in another.

30.2 Checkpoint policy

Checkpoints occur before risky file operations, after meaningful progress, before pause, at milestone completion, and before budget salvage. A checkpoint records Git state, uncommitted patch or commit, Claude session ID, workflow snapshot, active contract version, budget account versions, and last event sequence.

30.3 Database backup

The lab deployment performs encrypted daily full backups and frequent WAL or logical incremental backup to a separate device or NAS. At least one offline or physically separate copy is retained. Restore tests are scheduled, not assumed.

30.4 Artifact backup

Content-addressed artifacts are backed up independently of PostgreSQL. The database stores hashes and metadata; restoration verifies artifact integrity and reports missing objects.

30.5 Recovery objectives

Initial targets are:

- control-plane recovery point objective: 15 minutes or better;
- project repository recovery point: last checkpoint, normally under 10 minutes of meaningful work;
- service recovery time: under two hours for a trained administrator;
- no duplicate external side effect after recovery.

30.6 Provider outage

If OpenAI is unavailable, no new supervisor decision is made; Claude work pauses at a safe boundary. If Anthropic is unavailable, supervisors may continue discovery or review but no coding worker runs. qFoundry does not silently substitute a different coding provider in the baseline.

30.7 Node outage

A lost node lease marks work uncertain. The controller waits a bounded period, verifies the node is offline, then restores the last checkpoint to another compatible node. The previous node's lease is revoked so it cannot later submit valid events for the same execution.

30.8 Disk full

The node agent monitors free space and reserves emergency capacity. At warning threshold it blocks large tasks; at critical threshold it pauses workers and checkpoints to available storage. Raw logs are never allowed to consume all disk space.

30.9 Disaster recovery drill

Quarterly drills restore the database, artifacts, vault metadata, and one real project into an isolated environment, then verify event continuity, budget totals, and repository integrity.

Observability, Health, and Service Objectives

31.1 Health model

System health is not one green light. The dashboard reports database, workflow engine, model providers, node agents, event lag, credential broker, email, backups, and Sentinel separately. A summary state is derived from documented rules.

31.2 Service-level objectives

Initial operational objectives are:

Measure	Objective
Committed event to desktop visibility	95 percent within 2 seconds
Budget admission decision	99 percent within 250 ms
Permission policy decision	99 percent within 500 ms excluding human wait
API availability during scheduled operation	99.5 percent monthly
Project-state loss after acknowledged command	zero
Duplicate email/Git side effect after retry	zero in acceptance tests
Nightly backup success	99 percent, with immediate alert on failure

31.3 Logging

Logs are structured JSON with correlation IDs and classification. Sensitive request bodies are excluded by default. Debug logging has an expiration timer and may not be enabled globally on the POSM deployment without administrator action.

31.4 Traces

OpenAI Agents SDK traces and Claude/OpenTelemetry spans are correlated with qFoundry trace IDs. Provider traces are treated as auxiliary; the local event store remains the authoritative record. Trace export excludes secrets and can be disabled for sensitive projects.

31.5 Alerting

Alerts are generated for hard budget risk, reconciliation discrepancy, secret detection, sandbox denial spikes, workflow deadlock, node lease loss, event lag, failed backup, disk pressure, repeated model schema failure, and authentication anomalies.

31.6 Dashboard provenance

Every displayed metric has a source timestamp and status: live estimate, internally reconciled, or provider-authoritative. Clicking a cost, progress, or probability metric opens its calculation inputs, model version, and evidence.

Performance and Compute Sizing

32.1 Workload characteristics

Cloud providers perform large-model inference. Local compute is used for containers, builds, tests, browsers, databases, telemetry, statistical fitting, local utility models, and user interfaces. RAM, CPU cores, NVMe endurance, and reliability matter more initially than a top-tier GPU.

32.2 Per-worker planning envelope

A routine web/software worker should be planned at 4 CPU cores, 8–16 GB RAM, and 20–50 GB ephemeral disk. Heavy compiled, browser, EDA, or scientific projects may require 8–16 cores and 32–64 GB RAM. The scheduler uses project-specific profiles rather than assuming all workers are equal.

32.3 Initial dedicated machine

The recommended serious lab node is a Threadripper-class 24-core system with 128 GB ECC minimum and 256 GB preferred, multiple NVMe devices, separate backup, 2.5/10 GbE, and UPS. It should initially run four logical development pairs with two to four concurrent workers depending on workload.

32.4 Heterogeneous expansion

Donated Dell Precision, HP Z, Lenovo ThinkStation, or other x86 workstations can join as secondary nodes. qFoundry records architecture, cores, memory, disk, GPU, toolchain labels, and trust status for scheduling.

32.5 Raspberry Pi role

A Raspberry Pi is not a primary worker node because of limited RAM, ARM compatibility, and distributed-management overhead. A Pi is useful as an independent watchdog, UPS monitor, status display, Wake-on-LAN controller, or emergency-stop relay.

32.6 Local GPU

A GPU is optional for v0.1. It becomes valuable for local embeddings, small utility models, image/screenshot analysis, CUDA projects, and future offline assistance. GPU memory and software compatibility matter

more than gaming performance. The system shall operate fully without a local GPU.

32.7 Capacity admission

The node agent reports allocatable resources after reserving host capacity. The scheduler never allocates beyond a conservative fraction of physical RAM and CPU. The control plane, database, broker, and emergency operations retain reserved capacity even under high workload.

Test Strategy and Verification Gates

33.1 Test pyramid

The implementation uses unit tests for domain rules and algorithms, contract tests for provider and node adapters, integration tests for PostgreSQL/Temporal/broker behavior, end-to-end tests for user journeys, canary security tests, and controlled real-provider smoke tests with tiny budgets.

33.2 Fake providers

A deterministic fake OpenAI supervisor and fake Claude worker simulate streams, tool requests, schema errors, delays, cost usage, and interruptions. Most CI runs use fakes so the suite is fast and free. Recorded provider fixtures are sanitized and versioned.

33.3 Core capability gates

The release must prove:

1. supervisor session persistence and structured decisions;
2. Claude streaming, permission callback, interruption, and resume;
3. five consecutive closed-loop turns;
4. false-completion detection;
5. live event provenance;
6. budget reservation, threshold, salvage, and hard stop;
7. sandbox and cross-project isolation;
8. email reply correlation and idempotency;
9. crash recovery without duplicate side effects;
10. user takeover and return to supervisor;
11. multi-user authorization and one-active-project rule;
12. Sentinel detection of seeded anomalies;
13. completion of one real bounded project.

33.4 Budget tests

Property-based tests generate random sequences of reservations, completions, failures, and reconciliation updates and assert that the absolute ceiling is never exceeded by admitted worst-case obligations. Concurrent

reservation tests use many processes against the same account.

33.5 Temporal tests

Workflow tests advance virtual time through overnight waits, approval timeouts, retries, and retention expiration. Activities are fault injected before and after external side effects to verify idempotency.

33.6 Security tests

The canary repository attempts host reads, symlink escapes, secret printing, unauthorized network, dangerous package scripts, prompt injection, and protected-branch changes. Test success means containment and correct evidence, not merely that the worker “refused” in prose.

33.7 UI tests

Playwright validates golden scenarios, dense dashboard rendering, SSE reconnection, decision flows, budget details, and accessibility. Visual regression baselines cover the primary command center and execution stream at multiple window sizes.

33.8 Statistical tests

Sentinel models require simulation-based calibration where applicable, posterior predictive checks, chronological holdouts, calibration plots, drift tests, and model-card generation. A statistical model that fails diagnostics remains advisory or disabled.

Implementation Roadmap

34.1 Principle

The implementation builds vertical slices that prove the control loop, not isolated UI mockups or speculative ML. Each phase ends with executable evidence and a decision whether to proceed.

34.2 Phase 0 - Repository and foundations

Deliver monorepo, formatting, CI, typed contracts, PostgreSQL migrations, fake providers, event envelope, basic Tauri shell, and local development bootstrap.

34.3 Phase 1 - One closed loop

Deliver one project, one OpenAI supervisor, one Claude worker, one repository sandbox, streamed events, one permission callback, one budget, interrupt, checkpoint, and deterministic verification. This is the capability spike.

34.4 Phase 2 - Contract and evidence

Add discovery, versioned contract, compiler, milestones, requirement evidence, change requests, supervisor result review, and false-completion test.

34.5 Phase 3 - Durable operations

Move execution into Temporal workflows, implement crash recovery, idempotent Git/email activities, salvage mode, and session restoration.

34.6 Phase 4 - POSM multi-user

Add UMN OIDC approval, project memberships, master dashboard, one-active-project enforcement, per-user quotas, wallet scaffolding, and intake review.

34.7 Phase 5 - Full dashboard

Complete dense command center, live stream, code/diff view, terminal takeover, cost explorer, schedule, decision inbox, security page, and deliverable downloads.

34.8 Phase 6 - Portfolio execution

Support two then four active worker slots, worker-node protocol, resource scheduling, model routing, and dedicated lab deployment.

34.9 Phase 7 - Sentinel advisory

Implement immutable feature pipeline, baseline cost/time models, changepoint detection, progress-state advisory, model cards, and digital-twin simulation.

34.10 Phase 8 - Sentinel bounded authority

After validation, enable automatic pause for budget, secret, escape, stagnation, and telemetry-integrity conditions. Improvement experiments remain PR-only.

34.11 Phase 9 - Public release

Generalize identity and provider configuration, remove POSM-specific assumptions from engine defaults, publish AGPLv3 source, security policy, installer, operator guide, and migration path.

34.12 Milestone discipline

No phase begins solely because the calendar says so. The prior phase's acceptance evidence must pass. Features may be visually polished early, but governance and security gates are not deferred behind UI work.

Definition of Done and Release Acceptance

35.1 Subsystem definition of done

A subsystem is done only when:

- its public interfaces are schema documented;
- threat and failure cases are enumerated;
- unit and integration tests pass;
- telemetry and audit events exist;
- errors are actionable and classified;
- configuration and operational runbook are documented;
- performance and resource assumptions are measured;
- no unresolved critical or high security finding remains;
- contract requirements and evidence links are complete.

35.2 V0.1 acceptance

V0.1 is accepted only after a real project proceeds from idea through discovery, approved contract, intake, preflight quote, multiple supervisor-Claude cycles, live inspection, permission resolution, verification, bounded cost, deliverable package, and user acceptance. The process must survive at least one intentional application restart.

35.3 Financial acceptance

The live ledger must account for every provider call and display estimated versus reconciled amounts. Tiny-budget tests must prove provider caps, internal admission, salvage reserve, and absolute denial. Any unexplained reconciliation difference blocks release.

35.4 Security acceptance

The canary repository, cross-tenant authorization suite, secret-leak drill, backup restore, and protected-branch two-step confirmation must pass. Models cannot access host or provider credentials during tests.

35.5 Sentinel acceptance

Sentinel probabilities remain hidden from production decisions until each model has an approved model card and calibration evidence. The first release may show deterministic and descriptive analytics while probabilistic models are in shadow mode.

35.6 Human acceptance

The master operator must be able to understand current work, inspect raw evidence, stop any worker, take over a container, answer by dashboard or email, and export deliverables without consulting database tools.

35.7 Final release package

The release includes signed installer or reproducible build, container digests, database migration set, default policies, operator documentation, threat model, verification report, known limitations, and source commit identifier.

Part VI

Normative Implementation Appendices

Normative PostgreSQL Schema Excerpts

This appendix defines representative tables and constraints. Production migrations may split tables or add indexes, but may not weaken the stated invariants.

A.1 Tenants, users, and projects

```
1 CREATE TABLE tenants (  
2   id UUID PRIMARY KEY,  
3   slug TEXT NOT NULL UNIQUE,  
4   name TEXT NOT NULL,  
5   created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
6 );  
7  
8 CREATE TABLE users (  
9   id UUID PRIMARY KEY,  
10  tenant_id UUID NOT NULL REFERENCES tenants(id),  
11  email TEXT NOT NULL,  
12  status TEXT NOT NULL CHECK (status IN  
13    ('PENDING', 'ACTIVE', 'FROZEN', 'BANNED')),  
14  is_master BOOLEAN NOT NULL DEFAULT false,  
15  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
16  UNIQUE (tenant_id, email)  
17 );  
18  
19 CREATE TABLE projects (  
20   id UUID PRIMARY KEY,  
21   tenant_id UUID NOT NULL REFERENCES tenants(id),  
22   owner_user_id UUID NOT NULL REFERENCES users(id),  
23   name TEXT NOT NULL,  
24   state TEXT NOT NULL,  
25   state_version BIGINT NOT NULL DEFAULT 0,  
26   active_contract_version TEXT,  
27   workflow_id TEXT UNIQUE,  
28   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
29   updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
30 );  
31  
32 CREATE TABLE project_memberships (  
33   project_id UUID NOT NULL REFERENCES projects(id),  
34   user_id UUID NOT NULL REFERENCES users(id),  
35   role TEXT NOT NULL CHECK (role IN
```

```

36     ('OWNER', 'CONTRIBUTOR', 'REVIEWER', 'OBSERVER')),
37     PRIMARY KEY (project_id, user_id)
38 );
    
```

A.2 One active project per ordinary user

Because “active” spans multiple states, enforce the rule with a partial unique index over a materialized active-owner table or a transactionally maintained flag.

```

1  CREATE TABLE user_active_projects (
2      tenant_id UUID NOT NULL,
3      user_id UUID NOT NULL,
4      project_id UUID NOT NULL UNIQUE REFERENCES projects(id),
5      activated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
6      PRIMARY KEY (tenant_id, user_id)
7  );
    
```

Master administrators bypass insertion into this table; ordinary activation is transactional with the project-state transition.

A.3 Contracts and requirements

```

1  CREATE TABLE contracts (
2      id UUID PRIMARY KEY,
3      project_id UUID NOT NULL REFERENCES projects(id),
4      version TEXT NOT NULL,
5      status TEXT NOT NULL CHECK (status IN
6          ('DRAFT', 'USER_APPROVED', 'INTAKE_APPROVED', 'SUPERSEDED')),
7      content JSONB NOT NULL,
8      content_hash TEXT NOT NULL,
9      created_by UUID NOT NULL REFERENCES users(id),
10     created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
11     approved_at TIMESTAMPTZ,
12     UNIQUE (project_id, version),
13     UNIQUE (project_id, content_hash)
14 );
15
16 CREATE TABLE requirements (
17     id UUID PRIMARY KEY,
18     contract_id UUID NOT NULL REFERENCES contracts(id),
19     code TEXT NOT NULL,
20     title TEXT NOT NULL,
21     priority TEXT NOT NULL,
22     body JSONB NOT NULL,
23     UNIQUE (contract_id, code)
24 );
    
```

A.4 Budget ledger

```

1  CREATE TABLE budget_accounts (
2      id UUID PRIMARY KEY,
    
```

```

3  tenant_id UUID NOT NULL REFERENCES tenants(id),
4  user_id UUID REFERENCES users(id),
5  project_id UUID REFERENCES projects(id),
6  account_type TEXT NOT NULL,
7  currency TEXT NOT NULL DEFAULT 'USD',
8  target_amount NUMERIC(20,8),
9  absolute_ceiling NUMERIC(20,8),
10 version BIGINT NOT NULL DEFAULT 0,
11 created_at TIMESTAMPTZ NOT NULL DEFAULT now()
12 );
13
14 CREATE TABLE budget_entries (
15   id UUID PRIMARY KEY,
16   transaction_id UUID NOT NULL,
17   account_id UUID NOT NULL REFERENCES budget_accounts(id),
18   direction TEXT NOT NULL CHECK (direction IN ('DEBIT', 'CREDIT')),
19   amount NUMERIC(20,8) NOT NULL CHECK (amount >= 0),
20   entry_type TEXT NOT NULL,
21   reference_type TEXT NOT NULL,
22   reference_id TEXT NOT NULL,
23   pricing_version TEXT,
24   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
25   UNIQUE (account_id, reference_type, reference_id, entry_type)
26 );
27
28 CREATE TABLE budget_reservations (
29   id UUID PRIMARY KEY,
30   account_id UUID NOT NULL REFERENCES budget_accounts(id),
31   status TEXT NOT NULL CHECK (status IN
32     ('ACTIVE', 'SETTLED', 'RELEASED', 'EXPIRED')),
33   reserved_amount NUMERIC(20,8) NOT NULL,
34   provider_cap_amount NUMERIC(20,8) NOT NULL,
35   expires_at TIMESTAMPTZ NOT NULL,
36   idempotency_key TEXT NOT NULL UNIQUE,
37   created_at TIMESTAMPTZ NOT NULL DEFAULT now()
38 );

```

A.5 Events and outbox

```

1  CREATE TABLE project_sequences (
2   project_id UUID PRIMARY KEY REFERENCES projects(id),
3   next_sequence BIGINT NOT NULL
4  );
5
6  CREATE TABLE domain_events (
7   id UUID PRIMARY KEY,
8   tenant_id UUID NOT NULL,
9   project_id UUID,
10  sequence BIGINT,
11  event_type TEXT NOT NULL,
12  schema_version INTEGER NOT NULL,
13  occurred_at TIMESTAMPTZ NOT NULL,
14  recorded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
15  actor JSONB NOT NULL,
16  payload JSONB NOT NULL,

```

```

17  classification TEXT NOT NULL,
18  trace_id TEXT,
19  parent_event_id UUID,
20  previous_hash TEXT,
21  event_hash TEXT NOT NULL,
22  UNIQUE (project_id, sequence)
23 );
24
25 CREATE TABLE outbox (
26   id UUID PRIMARY KEY,
27   event_id UUID NOT NULL REFERENCES domain_events(id),
28   destination TEXT NOT NULL,
29   delivered_at TIMESTAMPTZ,
30   attempts INTEGER NOT NULL DEFAULT 0,
31   last_error TEXT
32 );
    
```

A.6 Permissions and decisions

```

1  CREATE TABLE permission_requests (
2   id UUID PRIMARY KEY,
3   project_id UUID NOT NULL REFERENCES projects(id),
4   task_id UUID,
5   tool_use_id TEXT,
6   action TEXT NOT NULL,
7   resource JSONB NOT NULL,
8   parameters_redacted JSONB NOT NULL,
9   risk_class TEXT NOT NULL,
10  status TEXT NOT NULL,
11  policy_rule_ids TEXT[] NOT NULL,
12  expires_at TIMESTAMPTZ,
13  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
14  resolved_at TIMESTAMPTZ
15 );
16
17 CREATE TABLE decisions (
18   id UUID PRIMARY KEY,
19   project_id UUID NOT NULL REFERENCES projects(id),
20   decision_type TEXT NOT NULL,
21   question TEXT NOT NULL,
22   options JSONB,
23   recommendation JSONB,
24   authority TEXT NOT NULL,
25   status TEXT NOT NULL,
26   response JSONB,
27   responded_by UUID REFERENCES users(id),
28   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
29   responded_at TIMESTAMPTZ
30 );
    
```

A.7 Row-level security example

```

1  ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
    
```



```
2
3 CREATE POLICY project_member_read ON projects
4 FOR SELECT USING (
5     tenant_id = current_setting('qf.tenant_id')::uuid
6 AND (
7     current_setting('qf.is_master')::boolean
8 OR EXISTS (
9     SELECT 1 FROM project_memberships pm
10    WHERE pm.project_id = projects.id
11    AND pm.user_id = current_setting('qf.user_id')::uuid
12 )
13 )
14 );
```

The database session context is set only by the authenticated API process. Direct public database access is prohibited.

Normative TypeScript Contracts

B.1 Opaque identifiers

```
1 type Brand<T, B extends string> = T & { readonly __brand: B };
2 type TenantId = Brand<string, "TenantId">;
3 type UserId = Brand<string, "UserId">;
4 type ProjectId = Brand<string, "ProjectId">;
5 type EventId = Brand<string, "EventId">;
6 type ContractHash = Brand<string, "ContractHash">;
```

Opaque IDs reduce accidental cross-entity substitution but do not replace authorization.

B.2 Money

```
1 interface Money {
2   currency: "USD";
3   // Canonical decimal with up to 8 fractional digits.
4   amount: string;
5 }
6
7 interface BudgetSnapshot {
8   accountId: string;
9   target?: Money;
10  absoluteCeiling: Money;
11  reconciled: Money;
12  pending: Money;
13  reserved: Money;
14  salvageProtected: Money;
15  safelyAvailable: Money;
16  version: number;
17 }
```

JavaScript floating point is not used for ledger arithmetic. The application uses a decimal library and database NUMERIC.

B.3 Project aggregate

```
1 type ProjectState =
```

```

2 | "DRAFT" | "DISCOVERY" | "CONTRACT_REVIEW"
3 | "READY" | "QUEUED" | "RUNNING" | "BLOCKED"
4 | "PAUSED" | "SALVAGE" | "TECHNICALLY_READY"
5 | "USER_ACCEPTED" | "CANCELLED" | "QUARANTINED";
6
7 interface ProjectSnapshot {
8     id: ProjectId;
9     tenantId: TenantId;
10    ownerUserId: UserId;
11    state: ProjectState;
12    version: number;
13    activeContract?: { version: string; hash: ContractHash };
14    currentMilestoneId?: string;
15    currentTaskId?: string;
16    waitingReason?: WaitingReason;
17    lastCheckpointId?: string;
18 }
    
```

B.4 Model request

```

1 interface CanonicalModelRequest {
2     requestId: string;
3     tenantId: TenantId;
4     projectId?: ProjectId;
5     purpose: "SUPERVISOR" | "WORKER" | "REVIEWER" |
6             "SENTINEL_EXPLANATION" | "UTILITY";
7     modelAlias: string;
8     messages: CanonicalMessage[];
9     tools?: CanonicalTool[];
10    outputSchema?: JsonSchema;
11    maxOutputTokens: number;
12    providerBudgetCap: Money;
13    traceContext: TraceContext;
14    dataClassification: "INTERNAL" | "SENSITIVE";
15 }
    
```

B.5 Normalized usage

```

1 interface NormalizedUsage {
2     provider: "OPENAI" | "ANTHROPIC" | "LOCAL";
3     providerRequestId?: string;
4     modelId: string;
5     inputTokens: bigint;
6     outputTokens: bigint;
7     cachedInputTokens?: bigint;
8     cacheWriteTokens?: bigint;
9     otherBillableUnits?: Record<string, string>;
10    estimatedCost: Money;
11    reconciledCost?: Money;
12    pricingVersion: string;
13    authoritative: boolean;
14 }
    
```

B.6 Node assignment

```

1  interface WorkerLease {
2      leaseId: string;
3      projectId: ProjectId;
4      taskId: string;
5      nodeId: string;
6      imageDigest: string;
7      repository: {
8          remote?: string;
9          baseCommit: string;
10         branch: string;
11     };
12     resources: {
13         cpuCores: number;
14         memoryBytes: bigint;
15         diskBytes: bigint;
16         maxProcesses: number;
17         wallClockSeconds: number;
18     };
19     networkProfileId: string;
20     issuedAt: string;
21     expiresAt: string;
22     signature: string;
23 }
    
```

B.7 Event contract

```

1  interface DomainEvent<T = unknown> {
2      eventId: EventId;
3      projectId?: ProjectId;
4      sequence?: number;
5      eventType: string;
6      schemaVersion: number;
7      occurredAt: string;
8      recordedAt: string;
9      actor: ActorRef;
10     payload: T;
11     cost?: CostDelta;
12     security: SecurityMetadata;
13     traceId?: string;
14     parentEventId?: EventId;
15     previousHash?: string;
16     eventHash: string;
17 }
    
```

B.8 Permission result

```

1  type PermissionOutcome = "ALLOW" | "DENY" | "SUPERVISOR" | "HUMAN";
2
3  interface PermissionEvaluation {
4      outcome: PermissionOutcome;
5      ruleIds: string[];
    
```

```
6   reasonCode: string;  
7   humanExplanation: string;  
8   sanitizedInput?: Record<string, unknown>;  
9   expiresAt?: string;  
10  evidenceEventIds: EventId[];  
11 }
```

Workflow Pseudocode

C.1 Project lifecycle

```
1 export async function projectLifecycle(input: ProjectInput) {
2   let state: ProjectWorkflowState = initialize(input);
3
4   while (!isTerminal(state.phase)) {
5     switch (state.phase) {
6       case "DISCOVERY":
7         state = await runDiscoveryUntilUserApproves(state);
8         break;
9
10      case "INTAKE":
11        state = await runSafetyResourceIntake(state);
12        break;
13
14      case "PREFLIGHT":
15        state.quote = await activities.generateQuote(state.projectId);
16        state = await waitForPlanSelectionOrEdit(state);
17        break;
18
19      case "QUEUED":
20        state.assignment = await activities.awaitSchedulerAdmission(state);
21        state.phase = "RUNNING";
22        break;
23
24      case "RUNNING":
25        state = await executeCurrentMilestone(state);
26        break;
27
28      case "BLOCKED":
29        state = await waitForDecisionPermissionOrDependency(state);
30        break;
31
32      case "PAUSED":
33        await condition(() => state.resumeRequested || state.cancelRequested);
34        state = handlePauseResolution(state);
35        break;
36
37      case "SALVAGE":
38        state = await runSalvage(state);
39        break;
```

```

40
41     case "TECHNICALLY_READY":
42         state = await waitForUserAcceptanceOrChangeRequest(state);
43         break;
44     }
45
46     if (historyShouldRotate(state)) {
47         return continueAsNew(snapshot(state));
48     }
49 }
50 return state;
51 }
    
```

C.2 Milestone loop

```

1  async function executeCurrentMilestone(state: ProjectWorkflowState) {
2      while (!milestoneTerminal(state)) {
3          const prompt = await activities.askSupervisorForNextAction(snapshot(state));
4
5          if (prompt.decision === "ASK_USER") {
6              return blockForUserDecision(state, prompt);
7          }
8          if (prompt.decision === "PAUSE") {
9              return transition(state, "PAUSED");
10         }
11
12         const reservation = await activities.reserveWorkerBudget({
13             projectId: state.projectId,
14             route: prompt.modelRoute,
15             taskId: state.taskId
16         });
17
18         if (!reservation.admitted) {
19             return transition(state, "SALVAGE");
20         }
21
22         const workerResult = await activities.runClaudeTurn({
23             prompt: prompt.workerPrompt,
24             reservationId: reservation.id,
25             sessionId: state.claudeSessionId
26         });
27
28         await activities.checkpointRepository(state.projectId);
29         const verification = await activities.runVerification(state.projectId);
30         state = await activities.applyEvidence(state, verification);
31
32         if (verification.securityBlock) return quarantine(state);
33         if (shouldSalvage(state)) return transition(state, "SALVAGE");
34     }
35     return transition(state, "TECHNICALLY_READY");
36 }
    
```

C.3 Permission handling

```
1  async function resolvePermission(req: PermissionRequest): Promise<Decision> {
2    const deterministic = await policy.evaluate(req);
3    if (deterministic.outcome === "ALLOW") return allow(deterministic);
4    if (deterministic.outcome === "DENY") return deny(deterministic);
5
6    if (deterministic.outcome === "SUPERVISOR") {
7      const review = await supervisor.reviewPermission(req, deterministic);
8      if (review.outcome !== "ESCALATE") return review;
9    }
10
11    await notifications.createDecision(req);
12    return await workflow.waitForHumanResolution(req.id, req.expiresAt);
13  }
```

C.4 Salvage

```
1  async function runSalvage(state: ProjectWorkflowState) {
2    await activities.stopStartingNewTools(state.projectId);
3    await activities.interruptAtSafeBoundary(state.projectId);
4    const checkpoint = await activities.checkpointRepository(state.projectId);
5    const tests = await activities.runCheapestUsefulVerification(state.projectId);
6    const report = await activities.generateDeterministicSalvageReport({
7      state, checkpoint, tests
8    });
9    await activities.queueSalvageNotification(report);
10   return transitionWithData(state, "PAUSED", { salvageReportId: report.id });
11 }
```

C.5 Idempotent email activity

```
1  async function sendEmailActivity(input: SendEmailInput) {
2    const existing = await db.emailSends.findByKey(input.idempotencyKey);
3    if (existing?.providerMessageId) return existing;
4
5    const draft = await renderAndRedact(input);
6    const response = await gmail.send(draft);
7
8    return await db.transaction(async (tx) => {
9      return tx.emailSends.insertOrReturnExisting({
10        idempotencyKey: input.idempotencyKey,
11        providerMessageId: response.id,
12        threadId: response.threadId
13      });
14    });
15 }
```


Event Catalog

D.1 Project and contract events

Event	Meaning
<code>project.created</code>	Draft project created.
<code>project.state.changed</code>	Validated lifecycle transition.
<code>discovery.message.recorded</code>	Human or supervisor discovery message persisted.
<code>contract.draft.generated</code>	Compiler created a candidate contract.
<code>contract.conflict.detected</code>	Static or supervisor conflict discovered.
<code>contract.user_approved</code>	User approved exact content hash.
<code>contract.intake_approved</code>	POSM intake accepted project for execution.
<code>change_request.opened</code>	Contract change proposed.
<code>change_request.approved</code>	User approved a new contract version.

D.2 Supervisor and worker events

Event	Meaning
<code>supervisor.run.started</code>	OpenAI supervisor invocation admitted.
<code>supervisor.decision.completed</code>	Structured decision validated.
<code>supervisor.policy.disagreement</code>	Proposed action conflicted with deterministic policy.
<code>worker.session.initialized</code>	Claude session and capabilities reported.
<code>worker.message.delta</code>	Partial visible worker response.
<code>worker.message.completed</code>	Worker message completed.
<code>worker.tool.requested</code>	Claude requested a tool action.
<code>worker.tool.completed</code>	Tool call completed with output reference.
<code>worker.turn.completed</code>	Claude query returned success result and usage.
<code>worker.turn.failed</code>	Query ended in error.
<code>worker.interrupted</code>	qFoundry interrupted worker execution.

D.3 Budget events

Event	Meaning
<code>budget.reservation.created</code>	Worst-case obligation reserved.
<code>budget.reservation.denied</code>	Insufficient safely available balance.
<code>budget.usage.estimated</code>	Live or returned estimated cost posted.
<code>budget.usage.reconciled</code>	Authoritative usage posted.
<code>budget.threshold.crossed</code>	Policy threshold entered.
<code>budget.provider_cap_reached</code>	Provider/SDK cap stopped query.
<code>budget.salvage.started</code>	Protected salvage sequence began.
<code>budget.reconciliation.discrepancy</code>	Internal/provider totals diverged.

D.4 Verification and progress events

Event	Meaning
<code>verification.started</code>	Verification plan began.
<code>test.completed</code>	Structured test result persisted.
<code>evidence.claim.proposed</code>	Worker/supervisor linked evidence to requirement.
<code>evidence.claim.validated</code>	Independent verifier accepted claim.
<code>evidence.claim.stale</code>	Later change invalidated claim.
<code>project.progress.meaningful</code>	Verified project progress occurred.
<code>milestone.technically_ready</code>	Milestone completion criteria satisfied.
<code>project.user_accepted</code>	User accepted final deliverable.

D.5 Security and infrastructure events

Event	Meaning
<code>security.secret.detected</code>	Possible secret entered monitored output.
<code>security.sandbox.denied</code>	Filesystem/process escape denied.
<code>security.network.denied</code>	Egress request blocked.
<code>security.incident.opened</code>	Incident record created.
<code>account.frozen</code>	User access immediately disabled.
<code>node.lease.lost</code>	Worker node failed to renew assignment.
<code>telemetry.integrity.failed</code>	Event chain or source validation failed.
<code>backup.completed</code>	Backup completed and verified.
<code>backup.failed</code>	Backup operation failed.

Permission Rules and Command Classes

E.1 Global invariants

Rule	Mandatory behavior
INV-001	No model sees raw long-lived provider or infrastructure credentials.
INV-002	No worker reads or writes outside assigned mounts.
INV-003	No AI pushes directly to or merges a protected branch.
INV-004	No admitted obligation exceeds the account absolute ceiling.
INV-005	No AI modifies its own policy, budget, or authority.
INV-006	Every external side effect is attributable and idempotent.
INV-007	Raw immutable telemetry cannot be modified by Sentinel.
INV-008	Cross-tenant data access is denied and treated as a security event.

E.2 Green actions

Green actions may be automatically allowed when they remain inside the project sandbox and match task scope:

- read approved workspace files;
- edit source and test files;
- run allowlisted build, lint, typecheck, and test commands;
- create local Git commits and branches;
- start bounded localhost development servers;
- install pre-approved locked dependencies;
- read official documentation through approved domains.

E.3 Yellow actions

Yellow actions require supervisor evaluation and may require human escalation:

- add a new dependency;
- modify CI, container, or migration configuration;
- delete files beyond a small threshold;
- access a new external API or domain;
- change database schema in a development environment;

- perform a major refactor while preserving contract behavior;
- push a feature branch or open a pull request when project permission exists.

E.4 Red actions

Red actions always require a human or are prohibited:

- access secrets or another project;
- increase budget or transfer funds;
- force push, merge, deploy, or alter protected settings;
- delete repository or large data sets;
- access private network devices;
- broaden project scope or change contract requirements;
- execute system administration commands;
- disable logging, sandboxing, or policy enforcement.

E.5 Command parsing examples

Command	Class	Result
<code>npm test - serializer</code>	local-test	Green if package script is approved.
<code>git status -short</code>	git-read	Green.
<code>git push origin feature/x</code>	git-remote	Yellow; brokered, project opt-in required.
<code>rm -rf src</code>	destructive-delete	Red/human, normally deny.
<code>curl https://unknown.example</code>	unknown-network	Yellow or deny.
<code>cat ~/.ssh/id_rsa</code>	host-secret	Invariant deny and incident.
<code>docker run -v /:/host ...</code>	host-control	Invariant deny.
<code>npm install useful-package</code>	dependency	Dependency review path.

Budget Algorithms and Numerical Examples

F.1 Fixed-point arithmetic

All amounts are parsed into decimal values with eight fractional digits. User display rounds to cents, but reservations and reconciliation retain greater precision.

F.2 Atomic reservation algorithm

```
1  async function reserve(input: ReserveInput): Promise<ReservationResult> {
2    return db.transaction({ isolation: "serializable" }, async (tx) => {
3      const account = await tx.accounts.lockForUpdate(input.accountId);
4      const totals = await tx.ledger.computeObligations(account.id);
5      const required = computeWorstCase(input.pricing, input.request);
6
7      const safeAvailable = account.absoluteCeiling
8        .minus(totals.reconciled)
9        .minus(totals.pending)
10       .minus(totals.activeReservations)
11       .minus(account.salvageReserve);
12
13      if (required.gt(safeAvailable)) {
14        return { admitted: false, required, safeAvailable };
15      }
16
17      const reservation = await tx.reservations.insert({
18        amount: required,
19        providerCap: min(required, input.providerSupportedCap),
20        idempotencyKey: input.idempotencyKey,
21        expiresAt: input.expiresAt
22      });
23      await tx.events.append(reservationCreated(reservation));
24      return { admitted: true, reservation };
25    });
26  }
```

F.3 Settlement

```

1  async function settle(reservationId: string, usage: NormalizedUsage) {
2      return db.transaction({ isolation: "serializable" }, async (tx) => {
3          const r = await tx.reservations.lockForUpdate(reservationId);
4          if (r.status === "SETTLED") return r.settlement;
5          if (usage.estimatedCost.gt(r.reservedAmount)) {
6              await tx.incidents.open("RESERVATION_UNDERRUN", { reservationId });
7          }
8          await tx.ledger.postUsage(r.accountId, usage);
9          await tx.reservations.markSettled(r.id, usage.estimatedCost);
10         await tx.events.append(usageSettled(r, usage));
11     });
12 }
    
```

F.4 Example: target with tolerance

Suppose target is \$20.00, tolerance \$0.50, absolute ceiling \$20.50, and salvage reserve \$0.60. Reconciled spend is \$18.72, pending usage \$0.09, and active reservations \$0.18. Safely available feature budget is

$$20.50 - 18.72 - 0.09 - 0.18 - 0.60 = \$0.91.$$

A proposed query bounded at \$0.74 is admitted. A query bounded at \$1.02 is denied even if its expected cost is only \$0.22.

F.5 Reservation utilization

Sentinel tracks $U = A/R$, actual divided by reserved. Persistently low utilization suggests overly conservative bounds; values near or above one indicate inadequate reservation. Tightening is allowed only after calibration and never at the expense of the absolute ceiling.

F.6 Project threshold state

Threshold percentage is computed against spendable budget excluding protected salvage:

$$q = \frac{S_{reconciled} + S_{pending} + S_{reserved}}{C_{absolute} - R_{salvage}}.$$

Threshold transitions emit events once per direction and policy version.

F.7 Provider cap mismatch

If a provider supports only an approximate cap, the internal reservation remains authoritative. The gateway also sets maximum output/turn controls and interrupts streams when internal pending usage reaches a conservative cutoff. Provider and internal caps are defense in depth.

Sentinel Model Specifications

G.1 Model registry record

Each statistical model version stores:

- decision purpose and forbidden uses;
- outcome and prediction horizon;
- training-window and inclusion criteria;
- feature schema/version;
- likelihood and prior specification;
- posterior diagnostics;
- holdout metrics and calibration;
- subgroup limitations;
- activation state and authority;
- code commit and environment lockfile.

G.2 Cost model

A hurdle model handles zero-cost deterministic tasks and positive-cost model tasks:

$$P(C_i > 0) = \text{logit}^{-1}(\eta_i),$$

$$\log C_i \mid C_i > 0 \sim \mathcal{N}(\mu_i, \sigma_{k[i]}).$$

Repository and task-category intercepts receive hierarchical priors. Model and effort-level effects are explicit. Posterior predictive intervals become quote intervals after conformal calibration if needed [3, 18].

G.3 Time model

Engineering active time and human wait time are modeled separately:

$$T_{calendar} = T_{active} + T_{queue} + T_{human} + T_{provider}.$$

The user sees both active-work and calendar forecasts. A Weibull accelerated-failure-time model is a candidate baseline; proportional-hazard assumptions are tested rather than assumed.

G.4 Progress-state emissions

Candidate emissions for the hidden semi-Markov model include:

- count and diversity of files read/edited;
- targeted versus full test outcomes;
- evidence claims created or invalidated;
- edit-revert similarity;
- repeated command signatures;
- token and dollar rate;
- tool denials and questions;
- elapsed active time since progress.

The model's inferred state is never used to override deterministic evidence completion.

G.5 Changepoint model

For metric stream x_t , Bayesian Online Changepoint Detection maintains

$$P(r_t \mid x_{1:t}),$$

where r_t is run length since the last change. Hazard priors are chosen per metric and evaluated on injected shifts [1].

G.6 Routing bandit

Context x contains task/repository features; action a is a model/review route; reward combines verified requirement value, cost, time, rework, and attention. Resource consumption includes dollars and model quotas. The policy begins as off-policy evaluation and shadow recommendations before any controlled exploration [2].

G.7 Calibration requirements

A probability may be displayed only if:

- the outcome is observable and sufficiently represented;
- calibration error is within the model-card threshold;
- the current input is not materially out of distribution;
- posterior diagnostics pass;
- the UI displays uncertainty and support class.

G.8 Model-card example

```

1 model_id: cost-hier-lognormal-v0.4
2 outcome: final_worker_cost_usd
3 horizon: current_task
4 training_tasks: 214
5 repositories: 31
6 validation: chronological_holdout
    
```



```
7 metrics:
8   interval_90_coverage: 0.887
9   median_absolute_error_usd: 0.42
10 limitations:
11   - sparse FPGA tasks
12   - no production deployment tasks
13 activation: advisory
14 forbidden_uses:
15   - automatic budget increase
16   - safety permission decisions
```

UI Screen and Interaction Specification

H.1 Portfolio command center

The home screen contains a persistent top system bar, project grid, decision inbox, schedule rail, alerts, and resource map. Project cards display state, milestone, current action, implemented/verified scope, deadline probability when calibrated, budget by actor, tests, last meaningful progress, and node assignment.

H.2 Project workspace

The project workspace has the following primary tabs:

1. Overview;
2. Live Execution;
3. Contract;
4. Roadmap and Schedule;
5. Code and Diffs;
6. Evidence and Verification;
7. Costs;
8. Decisions and Change Requests;
9. Permissions and Security;
10. Artifacts;
11. History and Checkpoints.

H.3 Live execution event card

Each event card displays timestamp, actor, event label, concise summary, cost delta, duration, and status. Expansion reveals full redacted input/output, command, files, tool identity, policy rule, trace, container, source commit, and provenance.

H.4 Cost explorer

The cost screen contains:

- target, tolerance, hard ceiling, and salvage reserve;
- supervisor, worker, reviewer, and Sentinel subaccounts;
- estimated, reserved, pending, reconciled, and available values;

- per-call table with model, tokens, pricing version, and status;
- burn forecast and quote intervals;
- cache usage and estimated savings;
- threshold history and salvage controls.

H.5 Decision inbox

Decisions are ordered by security severity, critical-path impact, budget impact, and age. Each card includes question, options, supervisor recommendation, evidence, consequence of delay, and required authority.

H.6 Sentinel screen

Sentinel displays findings, portfolio forecasts, model cards, calibration, experiments, anomalies, and recommended schedule changes. Raw probabilities are hidden when validation requirements are unmet. Every recommendation exposes its supporting model and telemetry window.

H.7 Visual system

The interface uses a near-black technical background, POSM photonic blue and cyan accents, thin borders, restrained glow, smooth transitions, and highly legible dense typography. Animation communicates live state but never hides values or delays emergency controls.

H.8 Accessibility

All controls are keyboard accessible. Status is encoded through text and icon in addition to color. Motion reduction is respected. Dense tables support screen-reader labels and export. Terminal and code views expose selectable text.

Detailed Repository Tree

```
1  qfoundry/  
2    apps/  
3      desktop/  
4        src-tauri/  
5          src/commands/  
6          capabilities/  
7          tauri.conf.json  
8        src/  
9          app/  
10         routes/  
11         features/  
12           portfolio/  
13           project-live/  
14           contract/  
15           costs/  
16           decisions/  
17           sentinel/  
18         components/  
19         state/  
20         api/  
21       control-api/  
22         src/  
23           routes/  
24           auth/  
25           application/  
26           projections/  
27           sse/  
28           websocket/  
29           middleware/  
30       workflow-worker/  
31         src/  
32           workflows/  
33           activities/  
34           signals/  
35           queries/  
36           versioning/  
37       model-gateway/  
38         src/  
39           providers/openai/  
40           providers/anthropic/  
41           catalog/  
42           admission/
```

```

43     redaction/
44     usage/
45     credential-broker/
46     src/
47     github/
48     gmail/
49     vault/
50     policy/
51     node-agent/
52     src/
53     leases/
54     containers/
55     worktrees/
56     commands/
57     artifacts/
58     telemetry/
59     sentinel-service/
60     qfoundry_sentinel/
61     features/
62     models/
63     calibration/
64     simulation/
65     experiments/
66     api/
67     packages/
68     domain/
69     src/entities/
70     src/state-machines/
71     src/services/
72     contracts/
73     src/api/
74     src/events/
75     src/provider/
76     src/node/
77     policy/
78     src/rules/
79     src/classifiers/
80     src/explain/
81     pricing/
82     catalogs/
83     src/accounting/
84     event-catalog/
85     schemas/
86     upcasters/
87     ui/
88     tokens/
89     components/
90     charts/
91     testkit/
92     fake-openai/
93     fake-claude/
94     canary-repos/
95     fixtures/
96     infra/
97     compose/
98     docker/images/
99     postgres/migrations/

```

```

100     temporal/
101     systemd/
102     windows-service/
103     tailscale/
104 docs/
105     adr/
106     api/
107     operations/
108     security/
    
```

I.1 Ownership

Each top-level module has a CODEOWNERS entry. Security-sensitive packages require a designated security reviewer. Database migrations and event-schema changes require control-plane and data-model review.

Initial Engineering Backlog

J.1 M0 - Foundation

ID	Deliverable	Evidence
M0-01	Create monorepo, CI, lint, typecheck, unit-test harness.	Clean pipeline on Windows and Linux.
M0-02	Implement domain IDs, money type, event envelope, schema generation.	Unit/property tests.
M0-03	Create PostgreSQL migration and local Temporal stack.	Migration/replay test.
M0-04	Create Tauri shell and authenticated API health view.	Packaged dev build.
M0-05	Implement fake OpenAI and fake Claude streaming adapters.	Deterministic fixture tests.

J.2 M1 - Closed-loop capability spike

ID	Deliverable	Evidence
M1-01	OpenAI supervisor structured decision adapter.	Schema and retry tests.
M1-02	Claude Agent SDK adapter with stream normalization.	Real tiny-budget smoke test.
M1-03	Deterministic permission callback and one escalation.	Allow/deny/interrupt tests.
M1-04	One sandboxed repository and command stream.	Host-access denial test.
M1-05	Live execution page with SSE reconnect.	Playwright golden scenario.

M1-06	Worker result verification and corrective supervisor turn.	Seeded incomplete-result test.
-------	--	--------------------------------

J.3 M2 - Contract and budget

ID	Deliverable	Evidence
M2-01	Discovery conversation and unresolved-question ledger.	Complete sample contract.
M2-02	Contract compiler and deterministic task packet.	Golden-output tests.
M2-03	Double-entry budget ledger and atomic reservation.	Concurrency property tests.
M2-04	Claude provider cap plus internal hard stop.	Tiny-cap test.
M2-05	Salvage workflow and report.	Forced threshold test.

J.4 M3 - Durability and integration

Implement Temporal workflows, checkpointing, GitHub App broker, Gmail escalation, restart recovery, artifacts, and two-step merge confirmation.

J.5 M4 - Multi-user POSM

Implement UMN OIDC approval, master role, project membership, one-active-project rule, quotas, intake review, and per-user wallet scaffolding.

J.6 M5 - Sentinel and portfolio

Implement feature store, model registry, baseline descriptive analytics, Bayesian cost model in shadow mode, changepoint detector, digital-twin simulation, two-worker scheduler, and bounded automatic pause rules.

Failure-Mode and Recovery Matrix

Failure	Detection	Automatic response	Human action
OpenAI timeout	Gateway timeout and no final usage	Release/expire reservation conservatively; retry policy	None unless repeated
Anthropic max budget	SDK error subtype and usage result	Checkpoint, settle usage, enter review/salvage	Add budget or accept handoff
Invalid supervisor schema	Output guardrail failure	Retry once with repair prompt, then pause	Review provider/config
Worker node lost	Lease expiration	Stop accepting events; restore checkpoint elsewhere	Inspect failed node
Container OOM	Runtime exit reason	Mark resource failure; do not blame tests; requote	Approve larger envelope
Disk pressure	Node metrics	Block large work, checkpoint, prune safe caches	Expand/clean storage
Database unavailable	Health and transaction failure	Stop command admission; retain worker at safe boundary	Restore database
Temporal unavailable	Worker/API health	No new workflow actions; running activity bounded by lease	Restore service
Event hash mismatch	Integrity verifier	Freeze affected project and event export	Security investigation
Secret detected	Scanner	Interrupt, quarantine output, revoke/rotate	Incident review
Unauthorized file read	Kernel/container denial	Interrupt or deny; create security finding	Review user intent
Unknown network domain	Egress proxy	Deny and create permission request	Approve temporary grant
Repeated test failure	Event heuristic	Supervisor diagnosis; after threshold pause	Optional intervention
Budget reconciliation gap	Ledger reconciliation	Freeze new calls if ceiling at risk	Provider/account review

Email duplicate delivery	Gmail history/idempotency	Ignore duplicate action	None
Email spoof/replay	Sender/token/replay check	Reject and log security event	Review if suspicious
Protected branch changed	Git provider event	Invalidate pending merge	Re-review exact SHA confirmation
Backup failed	Scheduled job result	Alert and retry to alternate target	Repair backup path
Sentinel model drift	Calibration/drift monitor	Revert to advisory/rules	Approve replacement model
Desktop disconnected	Transport timeout	Continue server work; client read-only cache	Reconnect
Power outage	UPS/node loss	Graceful shutdown if time; durable recovery	Restart and verify

K.1 Recovery principle

The correct recovery target is the last durable evidence-producing state, not necessarily the last token or terminal line. When uncertainty exists, the system favors duplicate-safe re-verification over assuming unrecorded work succeeded.

References

- [1] Ryan Prescott Adams and David J. C. MacKay. “Bayesian Online Changepoint Detection”. In: *arXiv preprint arXiv:0710.3742* (2007). URL: <https://arxiv.org/abs/0710.3742>.
- [2] Shipra Agrawal, Nikhil R. Devanur, and Lihong Li. “An Efficient Algorithm for Contextual Bandits with Knapsacks, and an Extension to Concave Objectives”. In: *Proceedings of the 29th Conference on Learning Theory*. Vol. 49. Proceedings of Machine Learning Research. 2016, pp. 4–18. URL: <https://proceedings.mlr.press/v49/agrawal16.html>.
- [3] Anastasios N. Angelopoulos, Stephen Bates, Michael I. Jordan, and Jitendra Malik. “Conformal Risk Control”. In: *International Conference on Learning Representations* (2024). URL: <https://arxiv.org/abs/2208.02814>.
- [4] Anthropic. *Claude Agent SDK Overview*. 2026. URL: <https://docs.anthropic.com/en/docs/claude-code/sdk> (visited on 06/14/2026).
- [5] Anthropic. *Claude Agent SDK Reference — TypeScript*. 2026. URL: <https://platform.claude.com/docs/en/agent-sdk/typescript> (visited on 06/15/2026).
- [6] Anthropic. *Claude Code LLM Gateway Configuration*. 2026. URL: <https://docs.anthropic.com/en/docs/claude-code/llm-gateway> (visited on 06/14/2026).
- [7] Anthropic. *Claude Code Monitoring and OpenTelemetry*. 2026. URL: <https://docs.anthropic.com/en/docs/claude-code/monitoring-usage> (visited on 06/14/2026).
- [8] A. Philip Dawid. “The Well-Calibrated Bayesian”. In: *Journal of the American Statistical Association* 77.379 (1982), pp. 605–610.
- [9] Docker. *Resource Constraints*. 2026. URL: https://docs.docker.com/engine/containers/resource_constraints/ (visited on 06/14/2026).
- [10] Docker. *Rootless Mode*. 2026. URL: <https://docs.docker.com/engine/security/rootless/> (visited on 06/14/2026).
- [11] Christian Fiedler et al. “On Safety in Safe Bayesian Optimization”. In: *arXiv preprint arXiv:2403.12948* (2024). URL: <https://arxiv.org/abs/2403.12948>.
- [12] Andrew Gelman et al. “Bayesian Workflow”. In: *arXiv preprint arXiv:2011.01808* (2020). URL: <https://arxiv.org/abs/2011.01808>.
- [13] GitHub. *Generating an Installation Access Token for a GitHub App*. 2026. URL: <https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/generating-an-installation-access-token-for-a-github-app> (visited on 06/14/2026).
- [14] GitHub. *Permissions Required for GitHub Apps*. 2026. URL: <https://docs.github.com/en/rest/authentication/permissions-required-for-github-apps> (visited on 06/14/2026).
- [15] Google. *Configure Push Notifications in the Gmail API*. 2026. URL: <https://developers.google.com/workspace/gmail/api/guides/push> (visited on 06/14/2026).
- [16] Google. *Implement Server-Side Authorization for Gmail API*. 2026. URL: <https://developers.google.com/workspace/gmail/api/auth/web-server> (visited on 06/14/2026).

- [17] Google. *Manage Threads in the Gmail API*. 2026. URL: <https://developers.google.com/workspace/gmail/api/guides/threads> (visited on 06/14/2026).
- [18] Bror Hultberg, Dave Zachariah, and Antônio H. Ribeiro. “Anytime-Valid Conformal Risk Control”. In: *arXiv preprint arXiv:2602.04364* (2026). URL: <https://arxiv.org/abs/2602.04364>.
- [19] National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1*. 2022. URL: <https://csrc.nist.gov/publications/detail/sp/800-218/final> (visited on 06/15/2026).
- [20] OpenAI. *Prompt Caching*. 2026. URL: <https://developers.openai.com/api/docs/guides/prompt-caching> (visited on 06/14/2026).
- [21] OpenAI. *Running Agents — OpenAI Agents SDK*. 2026. URL: https://openai.github.io/openai-agents-python/running_agents/ (visited on 06/15/2026).
- [22] OpenAI. *Sessions — OpenAI Agents SDK*. 2026. URL: <https://openai.github.io/openai-agents-python/sessions/> (visited on 06/15/2026).
- [23] OpenAI. *Tool Guardrails — OpenAI Agents SDK*. 2026. URL: https://openai.github.io/openai-agents-python/ref/tool_guardrails/ (visited on 06/15/2026).
- [24] OpenAI. *Tracing — OpenAI Agents SDK*. 2026. URL: <https://openai.github.io/openai-agents-python/tracing/> (visited on 06/14/2026).
- [25] OpenTelemetry Authors. *OpenTelemetry Specification*. 2026. URL: <https://opentelemetry.io/docs/specs/otel/> (visited on 06/15/2026).
- [26] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. 2025. URL: <https://genai.owasp.org/llm-top-10/> (visited on 06/15/2026).
- [27] PostgreSQL Global Development Group. *Row Security Policies*. 2026. URL: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (visited on 06/15/2026).
- [28] Yanan Sui, Alkis Gotovos, Joel Burdick, and Andreas Krause. “Safe Exploration for Optimization with Gaussian Processes”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research. 2015, pp. 997–1005. URL: <https://proceedings.mlr.press/v37/sui15.html>.
- [29] Tailscale. *Tailscale Serve*. 2026. URL: <https://tailscale.com/kb/1242/tailscale-serve> (visited on 06/15/2026).
- [30] Tauri Programme. *Tauri 2.0*. 2026. URL: <https://v2.tauri.app/> (visited on 06/14/2026).
- [31] Tauri Programme. *Tauri Capabilities*. 2026. URL: <https://v2.tauri.app/security/capabilities/> (visited on 06/14/2026).
- [32] Temporal Technologies. *Human-in-the-Loop AI Agent*. 2026. URL: <https://docs.temporal.io/ai-cookbook/human-in-the-loop-python> (visited on 06/14/2026).
- [33] Temporal Technologies. *Temporal Workflows*. 2026. URL: <https://docs.temporal.io/workflows> (visited on 06/14/2026).
- [34] Aki Vehtari, Andrew Gelman, and Jonah Gabry. “Practical Bayesian Model Evaluation Using Leave-One-Out Cross-Validation and WAIC”. In: *Statistics and Computing* 27 (2017), pp. 1413–1432. DOI: [10.1007/s11222-016-9696-4](https://doi.org/10.1007/s11222-016-9696-4).